

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

WEBSITE HỖ TRỢ SINH VIÊN NĂM NHẤT CÓ

TÍCH HỢP CHATBOT AI

Giảng viên hướng dẫn : Nguyễn Quang Hưng

Thành viên : Nguyễn Anh Huân - B21DCCN402
: Nguyễn Như Thiệu - B21DCCN690
: Vũ Anh Quân - B21DCCN618

Niên khoá : 2021 - 2026

Hệ đào tạo : Đại học chính quy

Hà Nội – 2025

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến các thầy cô của Học viện Công nghệ Bưu chính Viễn thông, đặc biệt là các thầy cô khoa Công nghệ thông tin đã trang bị cho chúng em những kiến thức quý báu trong suốt quá trình học tập tại học viện, tạo điều kiện thuận lợi nhất để chúng em hoàn thành đồ án.

Chúng em xin trân trọng gửi lời cảm ơn đặc biệt tới thầy giáo Nguyễn Quang Hưng, người đã hướng dẫn và giúp đỡ chúng em trong suốt quá trình thực hiện đồ án. Thầy sẵn sàng giải đáp và đề xuất hướng giải quyết khi chúng em gặp khó khăn để có thể hoàn thiện đồ án đúng tiến độ. Với điều kiện thời gian có hạn, lượng kiến thức để xây dựng đồ án rất rộng mà kinh nghiệm của chúng em còn hạn chế, đồ án này không thể tránh được những thiếu sót. Chúng em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của các thầy cô để chúng em có điều kiện bổ sung, nâng cao ý thức của mình, phục vụ tốt hơn công tác thực tế sau này.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày 28 tháng 12 năm 2025

Sinh viên thực hiện

Nguyễn Anh Huân

Nguyễn Như Thiệu

Vũ Anh Quân

NHẬN XÉT, ĐANH GIÁ, CHO ĐIỂM CỦA GIẢNG VIÊN PHẢN BIỆN

This image shows a full page of primary-ruled paper. It features approximately 28 horizontal rows, each defined by two parallel dotted lines. The paper is otherwise blank, with no margins, text, or other markings.

Điểm :.....(Bằng chữ:)

Hà Nội, ngày tháng năm 202...

Giảng viên hướng dẫn

Nguyễn Quang Hưng

MỤC LỤC

CHƯƠNG I : Khảo sát và ý tưởng.....	9
1.1 Khảo sát một số vấn đề của sinh viên:	9
1.1.1 Vấn đề tiếp cận thông báo	9
1.1.2 Vấn đề thay đổi môi trường và học tập	12
1.2 Mục tiêu của đề tài:	13
1.2.1 Xây dựng cổng thông tin tập trung và cộng đồng (Forum) để sinh viên hỗ trợ nhau.....	13
1.2.2 Phát triển Chatbot AI hỗ trợ giải đáp thắc mắc 24/7 dựa trên dữ liệu thực tế.....	14
CHƯƠNG II: Cơ sở lý thuyết và Công nghệ.....	16
2.1 Công nghệ phát triển Web:.....	16
2.1.1 Kiến trúc Monolithic	16
2.1.2 Công nghệ xây dựng Frontend	17
2.1.3 Công nghệ xây dựng Backend.....	18
2.2 Kỹ thuật RAG và Xử lý ngôn ngữ tự nhiên để làm chatbot	21
2.2.1 Giới thiệu về LLM (Large Language Models).....	21
2.2.2 Giới thiệu về kỹ thuật RAG.....	22
2.2.3 Tổng quan về RAG.....	22
2.2.4 Lợi ích của việc sử dụng RAG:	23
2.2.5 Vector đặc trưng	23
2.2.6 Vector Database.....	23
2.2.7 Triển khai RAG	26
CHƯƠNG III: Phân tích và Thiết kế hệ thống (System Analysis & Design)	28
3.1 Phân tích yêu cầu (Functional Requirements):	28
3.1.1 Xác định các tác nhân của hệ thống :	28
3.1.2 Xác định các yêu cầu chức năng	29
3.2 Biểu đồ Usecase tổng quát:	30
3.3 Thiết kế Cơ sở dữ liệu (ERD):	31
3.4 Thiết kế Kiến trúc Hệ thống :.....	34
3.4.1 Kiến trúc Frontend.....	34

3.4.2	Thiết kế Bảo mật	36
3.4.3	Kiến trúc Thời gian thực	38
Chương IV: Xây dựng Module Dữ liệu và AI Chatbot		40
4.1	Giới thiệu.....	40
4.2	Module thu thập dữ liệu	40
4.2.1	Thu thập dữ liệu từ forum.....	40
4.2.2	Thu thập dữ liệu từ các file đính kèm thông báo đã gửi	41
4.2.3	Dữ liệu thu thập từ website crawl từ trang chính thức của trường	42
4.3	Module AI Chatbot.....	42
4.3.1	Luồng hoạt động tổng thể của RAG.....	42
4.3.2	Chi tiết luồng hoạt động RAG.....	43
4.3.3	Đánh giá độ chính xác và các mô hình LLM đã thử nghiệm .	52
CHƯƠNG V: Cài đặt và Triển khai (Implementation).....		57
5.1	Cài đặt dự án :.....	57
5.1.1	Frontend:.....	57
5.1.2	Backend:	58
5.2	Giao diện và Chức năng chính (Kèm hình ảnh):.....	61

DANH MỤC HÌNH

Hình 1-1: Trang thông báo của Học viện.....	9
Hình 1-2: Trang thông báo của phòng Giáo vụ	10
Hình 1-3: Trang thông báo của phòng Đào tạo.....	10
Hình 1-4: Trang thông báo thông tin sinh viên.....	11
Hình 1-5: Trang facebook của Học viện.....	11
Hình 1-6: Trang facebook của Khoa CNTT - PTIT	11
Hình 1-7: Trang facebook Tôi yêu PTIT	12
Hình 1-8: Thư viện PTIT	13
Hình 2-1 : Giới thiệu RAG.....	22
Hình 2-2 : Giới thiệu Pinecone	25
Hình 2-3 : Giới thiệu Flowise	26
Hình 3-1: Biểu đồ Usecase tổng quát.....	30
Hình 3-2: Các bảng cho xác thực, phân quyền	31
Hình 3-3: Các bảng cho chức năng thông báo	31
Hình 3-4: Các bảng cho chức năng thư viện.....	32
Hình 3-5: Các bảng cho chức năng tính điểm.....	33
Hình 3-6: Các bảng cho chức năng forum	33
Hình 3-7: Các bảng cho chức năng quản lý log, thông báo	33
Hình 3-8: Quy trình xử lý JWT	36
Hình 3-9: Kiến trúc thời gian thực	38
Hình 4-1: Mô hình thu thập dữ liệu	40
Hình 4-2: Hình ảnh minh họa 2 bài post thu thập được từ forum.....	41
Hình 4-3: Hình ảnh văn bản PDF được convert từ ảnh chụp	41
Hình 4-4: Luồng hoạt động của hệ thống RAG	43
Hình 4-5: Hình ảnh minh họa chia tài liệu thành các chunk.....	44
Hình 4-6: Bản ghi dữ liệu mẫu.....	49
Hình 4-7: Ảnh minh họa câu hỏi với chatbot AI	50
Hình 4-8: Ảnh minh họa source lấy từ vector database để làm thông tin cho AI trả lời	51
Hình 4-9: Hình ảnh minh họa lịch sử cuộc trò chuyện	53
Hình 4-10: Hình ảnh minh họa kết quả đánh giá	54
Hình 4-11: Hình ảnh minh họa các model của gemini	55
Hình 5-1: Minh họa cài đặt docker desktop	58
Hình 5-2: Minh họa clone project	59
Hình 5-3: Minh họa chạy containers	60
Hình 5-4: Minh họa cài đặt database	60
Hình 5-5: Chạy dự án	61
Hình 5-6: Giao diện chatbot AI.....	61
Hình 5-7: Trang chính người dùng	61
Hình 5-8: Giao diện các tính năng	62

Hình 5-9: Giao diện xem bài đăng trong topic	62
Hình 5-10: Giao diện xem thông báo	63
Hình 5-11: Giao diện tính điểm	63
Hình 5-12: Giao diện thư viện tài liệu	64
Hình 5-13: Giao diện sự kiện & hoạt động	64
Hình 5-14: Giao diện lịch sự kiện	65
Hình 5-15: Giao diện admin dashboard	65
Hình 5-16: Giao diện quản lý thông báo	66
Hình 5-17: Giao diện quản lý diễn đàn	66

DANH MỤC BẢNG

Bảng 3-1: Tác nhân hệ thống chính	28
Bảng 3-2: Tác nhân hệ thống nghiệp vụ	29
Bảng 4-1: Bảng so sánh các mô hình đã chọn	56

LỜI MỞ ĐẦU

Trong cuộc đời một phần không nhỏ trong chúng ta, lên đại học là một quãng thời gian năng động, nhiệt huyết và đáng nhớ, bên cạnh đó, nó cũng là bước chuyển vô cùng quan trọng, rời xa vòng tay gia đình, rời xa sự an toàn khi ở trong môi trường quen thuộc, chúng ta có lẽ sẽ đến với những thành phố xa lạ, tiếp xúc những con người đến từ rất nhiều nơi. Có lẽ do vậy, dù hoạt bát, năng động đến đâu, ai trong chúng ta cũng không tránh khỏi những lúc không biết mình cần làm gì cho đúng, cần làm gì để hòa nhập với những người bạn mới, với môi trường còn xa lạ.

Một số sẽ thấy lạ lẫm với cách học ở đại học, không còn ai nhắc nhở, không còn những người bạn thân thuộc để cùng nhau trao đổi bài. Một số thì thấy khó khăn khi các thông báo, các nội quy của trường nhưng ở nhiều phòng ban khác nhau, nhiều trang web, fanpage khiến ta không thể kịp thời cập nhật mọi thứ. Một số khác cần lắm những chia sẻ, góp ý từ các thầy cô, anh chị, hay thậm chí là bạn bè xung quanh để có thể tìm ra cho mình những thông tin phù hợp nhất.

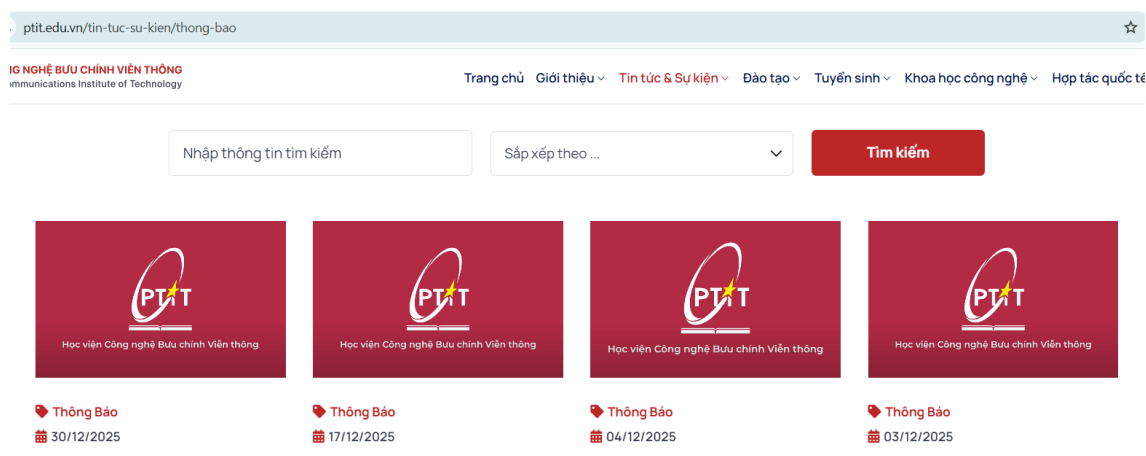
Nhận thấy những thách thức trên, nhóm chúng em quyết định chọn đề tài “Website hỗ trợ sinh viên năm nhất tích hợp chatbot AI” làm đồ án tốt nghiệp. Đề tài này nhằm mục tiêu phát triển một website tiện lợi và phù hợp với nhu cầu thực tế của sinh viên, đặc biệt là các sinh viên năm nhất, những người còn bỡ ngỡ với trang mới của cuộc sống này. Trang web vừa là nơi tổng hợp các thông báo từ nhà trường, vừa là nơi tìm kiếm tài liệu học tập, vừa là môi trường để các bạn sinh viên có thể trao đổi về các chủ đề cụ thể từ học tập đến đời sống. Và đặc biệt hơn cả là có thể hỏi đáp cùng AI để nhanh chóng tìm được thông tin hữu ích đối với bản thân.

CHƯƠNG I : Khảo sát và ý tưởng

1.1 Khảo sát một số vấn đề của sinh viên:

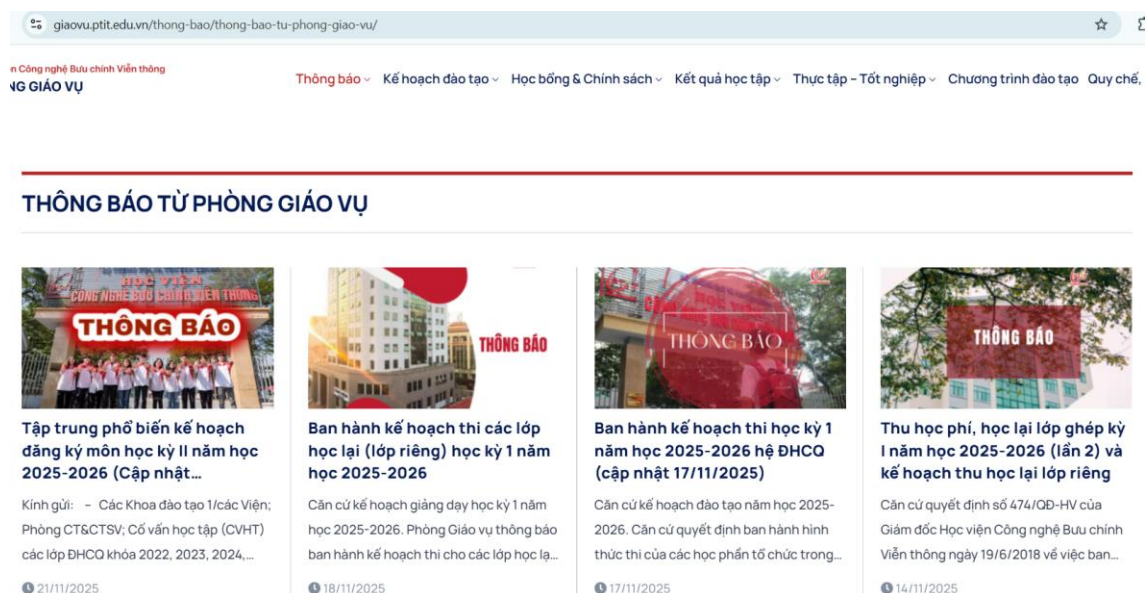
1.1.1 Vấn đề tiếp cận thông báo

Hiện nay, để tiếp cận các thông báo của Học viện Công nghệ Bưu chính Viễn thông, sinh viên phải theo dõi nhiều nguồn thông tin khác nhau. Các thông báo chính thức của Học viện được đăng tải trên trang web của trường, tiêu biểu như cổng thông tin thông báo tại địa chỉ <https://ptit.edu.vn/tin-tuc-su-kien/thong-bao>, nơi cung cấp các thông tin chung liên quan đến học tập, hành chính và các hoạt động của nhà trường.



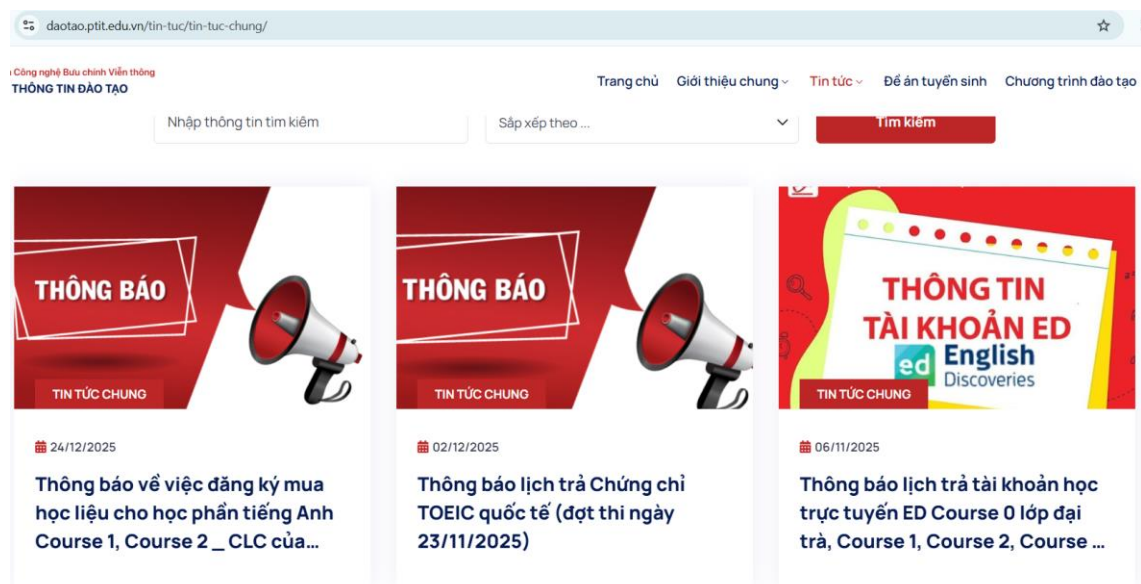
Hình 1-1: Trang thông báo của Học viện

Bên cạnh đó, các phòng ban chức năng trong Học viện cũng có những kênh thông báo riêng, đặc biệt là Phòng Giáo vụ – đơn vị trực tiếp quản lý các vấn đề học tập của sinh viên <https://giaovu.ptit.edu.vn/thong-bao/thong-bao-tu-phong-giao-vu/>.

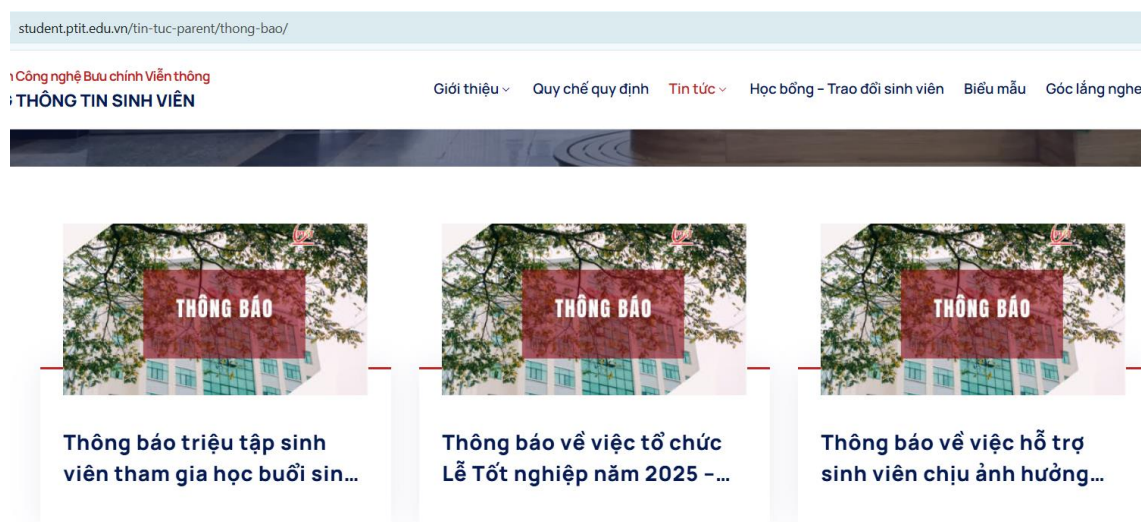


Hình 1-2: Trang thông báo của phòng Giáo vụ

Ngoài ra, một số cổng thông tin khác như <https://daotao.ptit.edu.vn/tin-tuc/tin-tuc-chung/> và <https://student.ptit.edu.vn/tin-tuc-parent/thong-bao/> cũng thường xuyên cập nhật các thông báo liên quan đến đào tạo và sinh viên.

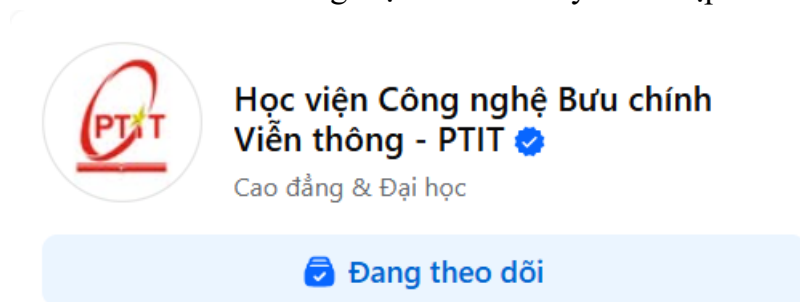


Hình 1-3: Trang thông báo của phòng Đào tạo

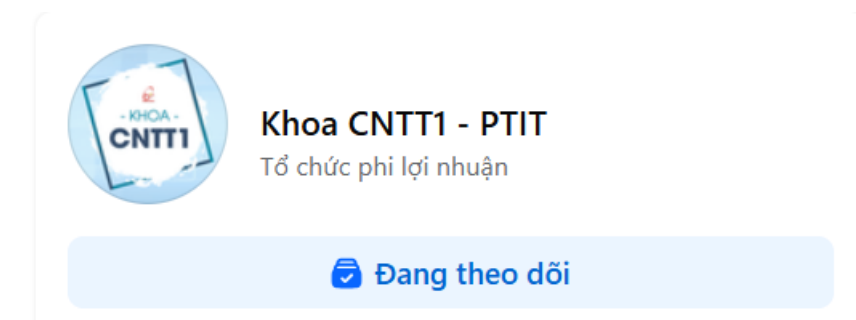


Hình 1-4: Trang thông báo thông tin sinh viên

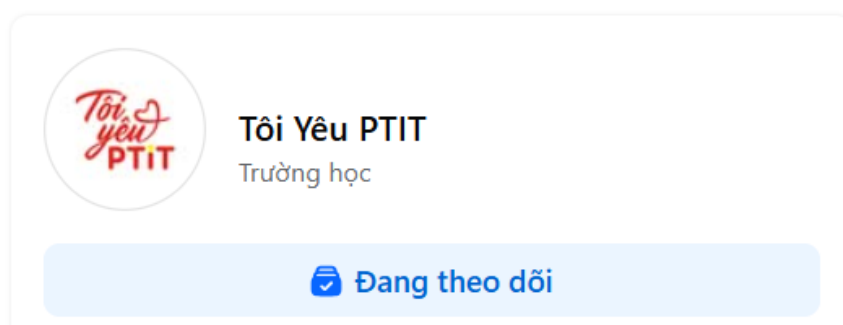
Không chỉ giới hạn ở các trang web chính thức, sinh viên còn tiếp cận thông tin thông qua các fanpage và hội nhóm trên mạng xã hội như “Học viện Công nghệ Bưu chính Viễn thông – PTIT”, “Tôi yêu PTIT”, “Khoa CNTT1 – PTIT”,... Điều này cho thấy hệ thống thông tin hiện tại đang tồn tại nhiều kênh phân tán, gây khó khăn cho sinh viên trong việc theo dõi đầy đủ và kịp thời các thông báo quan trọng.



Hình 1-5: Trang facebook của Học viện



Hình 1-6: Trang facebook của Khoa CNTT - PTIT



Hình 1-7: Trang facebook Tôi yêu PTIT

1.1.2 Vấn đề thay đổi môi trường và học tập

Giai đoạn chuyển tiếp từ phổ thông lên đại học đặt ra những thách thức lớn đối với sinh viên năm nhất (Freshmen), thường được mô tả là trạng thái "**Sốc văn hóa học đường**".

Sự thay đổi phương pháp đào tạo - Đây là rào cản rất lớn. Tại bậc phổ thông, học sinh quen thuộc với phương pháp học tập thụ động và sự giám sát chặt chẽ. Ngược lại, môi trường đại học yêu cầu năng lực **tự học (self-study)**, tư duy phản biện và khả năng tự nghiên cứu.

Việc thiếu hụt các kỹ năng này khiến nhiều tân sinh viên gặp khó khăn trong quá trình tiếp cận giáo trình, ghi chép bài giảng, tổng hợp kiến thức và xây dựng lộ trình học tập phù hợp. Hệ quả là kết quả học tập trong học kỳ đầu thường không đạt được kỳ vọng. Không phải sinh viên nào cũng có khả năng tự học hoặc tự tìm kiếm tài liệu học tập một cách hiệu quả, đặc biệt trong bối cảnh nguồn tài liệu trên Internet đa dạng nhưng phân tán, không đồng nhất và khó đảm bảo tính chính thống cho từng môn học cụ thể.

Trên thực tế, nhà trường và giảng viên đã cung cấp nhiều tài liệu học tập trong quá trình giảng dạy. Tuy nhiên, đối với những sinh viên có nhu cầu chủ động tìm hiểu trước nội dung môn học hoặc mở rộng kiến thức ngoài chương trình, việc xác định nguồn tài liệu phù hợp vẫn là một thách thức. Mặc dù thư viện trường và các hệ thống thư viện số cung cấp phong phú tài liệu thuộc nhiều lĩnh vực khác nhau, sinh viên thường gặp khó khăn trong việc xác định chính xác tài liệu nào phù hợp với từng môn học, nội dung và mục tiêu học tập cụ thể. Điều này cho thấy nhu cầu về một phương thức tìm kiếm và quản lý tài liệu hiệu quả hơn, trực quan và dễ tiếp cận hơn.



Hình 1-8: Thư viện PTIT

Bên cạnh vấn đề học tập, sinh viên năm nhất còn có nhu cầu trao đổi thông tin liên quan đến đời sống sinh viên như tìm kiếm chỗ ở, phương tiện đi lại, địa điểm ăn uống hoặc các hoạt động ngoại khóa. Hiện nay, các hội nhóm trên mạng xã hội, đặc biệt là Facebook, đóng vai trò là kênh trao đổi thông tin phổ biến. Tuy nhiên, các nhóm này thường chứa nhiều chủ đề khác nhau, thông tin thiếu cấu trúc, khó kiểm soát và gây khó khăn trong việc tìm kiếm hoặc tra cứu lại những nội dung cần thiết.

1.2 Mục tiêu của đề tài:

1.2.1 Xây dựng cổng thông tin tập trung và cộng đồng (Forum) để sinh viên hỗ trợ nhau.

- **Xây dựng trang tổng hợp thông báo:**

Hệ thống cung cấp trang tổng hợp thông báo, cho phép thu thập và hiển thị các thông báo từ các website chính thức của Học viện và các phòng ban liên quan. Nhờ đó, sinh viên có thể theo dõi thông tin một cách tập trung, nhanh chóng và thuận tiện. Bên cạnh việc tổng hợp thông báo, hệ thống còn hỗ trợ tạo và quản lý các thông báo mới trực tiếp trên nền tảng, đồng thời phân phối thông tin đến các nhóm sinh viên liên quan.

- **Xây dựng trang thư viện tài liệu:**

Hệ thống xây dựng thư viện tài liệu học tập được tổ chức theo kỳ học, khoa và từng môn học cụ thể, giúp sinh viên dễ dàng tìm kiếm và tiếp

cận tài liệu phù hợp với nhu cầu học tập. Ngoài giáo trình và tài liệu tham khảo, hệ thống còn tổng hợp đề thi các năm của từng môn học, hỗ trợ sinh viên trong quá trình ôn tập và đánh giá năng lực.

- **Xây dựng trang tính điểm CPA:**

Trang hỗ trợ tính toán điểm trung bình tích lũy (CPA), điểm trung bình học kỳ (GPA) cho sinh viên dựa trên danh sách môn học theo từng học kỳ. Hệ thống cho phép sinh viên nhập hoặc cập nhật điểm các môn học, từ đó tự động tính toán kết quả tổng kết. Thông qua công cụ này, sinh viên có thể xác định mục tiêu học tập, mức điểm cần đạt cho từng môn nhằm đạt các danh hiệu học tập mong muốn, cũng như hỗ trợ lập kế hoạch học cải thiện hoặc học lại khi cần thiết.

- **Xây dựng một Forum hỏi đáp:**

Hệ thống cung cấp diễn đàn trao đổi thông tin thông qua các danh mục (Category) và chủ đề (Topic) rõ ràng, tạo môi trường giao lưu và chia sẻ kiến thức có tổ chức. Diễn đàn hỗ trợ sinh viên thảo luận về các vấn đề học tập, lớp học, câu lạc bộ và các hoạt động liên quan đến đời sống sinh viên, giúp thông tin được lưu trữ, tìm kiếm và tra cứu một cách hiệu quả.

1.2.2 Phát triển Chatbot AI hỗ trợ giải đáp thắc mắc 24/7 dựa trên dữ liệu thực tế.

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, đặc biệt là sự bùng nổ của trí tuệ nhân tạo (AI), các hệ thống chatbot thông minh như ChatGPT, Gemini,... ngày càng được sử dụng rộng rãi nhằm hỗ trợ người dùng tìm kiếm thông tin và giải đáp thắc mắc một cách nhanh chóng.

Đối với sinh viên, đặc biệt là sinh viên năm nhất trong giai đoạn đầu làm quen với môi trường đại học, nhu cầu được hỗ trợ thông tin là rất lớn. Các câu hỏi liên quan đến vị trí các phòng ban, nội quy – quy định của Nhà trường, quy trình học tập, cũng như thông tin về các câu lạc bộ và hoạt động ngoại khóa là những vấn đề thiết thực giúp sinh viên nhanh chóng hòa nhập và tự tin hơn trong môi trường học tập mới.

Xuất phát từ thực tế đó, đề tài đề xuất xây dựng một hệ thống chatbot AI nhằm hỗ trợ trả lời các câu hỏi liên quan đến Học viện Công nghệ Bưu chính Viễn thông. Chatbot được xây dựng dựa trên dữ liệu thu thập từ nhiều nguồn như :

- Các website chính thức của Học viện, các fanpage, hội nhóm sinh viên PTIT.
- Đồng thời được cập nhật và đồng bộ từ hệ thống website được thiết kế trong đề tài.

Thông qua đó, hệ thống góp phần giúp sinh viên tiếp cận thông tin một cách nhanh chóng, chính xác và hiệu quả hơn.

CHƯƠNG II: Cơ sở lý thuyết và Công nghệ

2.1 Công nghệ phát triển Web:

2.1.1 Kiến trúc Monolithic

Monolithic (Kiến trúc nguyên khối) là ứng dụng mọi dịch vụ đều nằm trên một hệ thống và dùng chung một database. Người dùng thao tác trực tiếp với giao diện, giao diện gọi đến chức năng, chức năng tìm đến dữ liệu trong database.

- Ưu điểm:
 - Đơn giản hóa quá trình vận hành và phát triển: Việc tích hợp các thành phần và chức năng vào một đơn vị duy nhất giúp đơn giản hóa quá trình xây dựng và phát triển ứng dụng. Bằng cách này, các lập trình viên có thể phát triển ứng dụng một cách nhanh chóng và tiết kiệm thời gian hơn.
 - Khả năng mở rộng dễ dàng: Mô hình Monolithic cho phép mở rộng quy mô và triển khai thêm nhiều phiên bản của ứng dụng một cách thuận tiện. Điều này giúp các nhà phát triển có thể tăng cường khả năng chịu tải và xử lý của hệ thống một cách dễ dàng.
 - Hiệu suất tối ưu: Vì ứng dụng hoạt động như một hệ thống duy nhất, không có overhead của việc gọi qua lại giữa các thành phần khác nhau, Monolithic thường có hiệu suất tốt hơn so với các mô hình phân tán khác.
- Nhược điểm:
 - Sự phụ thuộc cao: Monolithic có mức độ phụ thuộc mạnh giữa các thành phần trong hệ thống. Khi một phần gặp lỗi, nó có thể ảnh hưởng đến toàn bộ ứng dụng và gây gián đoạn hoạt động của hệ thống. Điều này cũng khiến việc bảo trì hệ thống trở nên khó khăn và phức tạp hơn.
 - Khó mở rộng: Theo thời gian, khi dự án trở nên phức tạp hơn đòi hỏi ứng dụng liên tục phát triển và mở rộng quy mô. Việc này trở nên khó khăn bởi nó đòi hỏi nhiều công sức và tài nguyên, thậm chí là phải “xây” lại từ đầu.
 - Kém linh hoạt: Monolithic không linh hoạt trong việc áp dụng công nghệ, kỹ thuật mới do nhiều ứng dụng nguyên khối thường phụ thuộc một công nghệ cũ và lỗi thời. Để sử dụng các công nghệ mới, cần phải thay đổi toàn bộ ứng dụng, gây rủi ro và tốn kém về thời gian và nguồn lực.

- Lãng phí tài nguyên: Là một ứng dụng nguyên khối lớn, Monolithic sẽ mất nhiều thời gian để khởi động và khi một service scale dẫn đến toàn bộ ứng dụng phải scale theo. Điều này tiêu tốn nhiều tài nguyên CPU và bộ nhớ.

2.1.2 Công nghệ xây dựng Frontend

a. Giới thiệu về React JS

- ReactJS là một thư viện JavaScript mã nguồn mở, chủ yếu được sử dụng để xây dựng các giao diện người dùng cho các ứng dụng web một cách hiệu quả và dễ dàng. React cho phép phát triển các ứng dụng một trang (Single Page Application) với cấu trúc động, phản hồi nhanh và dễ bảo trì.
- React sử dụng khái niệm component (thành phần) để chia nhỏ giao diện người dùng thành các đơn vị tái sử dụng được, giúp tăng tính modular và dễ dàng quản lý mã nguồn. Thư viện này nổi bật nhờ vào khả năng render nhanh chóng và cập nhật giao diện một cách hiệu quả thông qua cơ chế Virtual DOM.

b. Nguyên lý hoạt động của React JS

Virtual DOM (DOM ảo) : React sử dụng một phiên bản nhẹ của DOM gọi là Virtual DOM. Khi có thay đổi trong trạng thái của ứng dụng (state), React sẽ cập nhật Virtual DOM trước, sau đó so sánh với DOM thật để chỉ cập nhật những phần thay đổi, giúp tối ưu hiệu suất và giảm bớt thao tác với DOM, vốn có thể rất tốn thời gian.

Component-based architecture : Dữ liệu trong React được truyền theo một hướng duy nhất: từ component cha (parent component) xuống component con (child component) thông qua props. Đây là mô hình dữ liệu một chiều (unidirectional data flow), giúp dễ dàng theo dõi sự thay đổi của dữ liệu và ngăn chặn những lỗi phức tạp do việc quản lý trạng thái không rõ ràng.

State và Props :

- State là nơi lưu trữ dữ liệu nội bộ của mỗi component và có thể thay đổi theo thời gian. Khi state thay đổi, React sẽ tự động re-render lại component để cập nhật giao diện.
- Props là cơ chế truyền dữ liệu từ component cha xuống component con, nhưng props là read-only và không thể thay đổi trong component con.

Reconciliation (So khớp): React sử dụng thuật toán reconciliation để so sánh Virtual DOM và DOM thật, từ đó xác định phần nào trong

giao diện cần được cập nhật. Quá trình này rất nhanh nhờ vào việc chỉ thay đổi những phần thực sự cần thiết, giúp tối ưu hiệu suất khi ứng dụng phức tạp.

JSX (JavaScript XML) : React sử dụng JSX, một cú pháp giống như HTML nhưng thực chất là JavaScript. JSX cho phép bạn mô tả giao diện người dùng trực tiếp trong mã JavaScript, giúp tăng tính dễ đọc và dễ viết mã. JSX sẽ được chuyển thành mã JavaScript thuần thông qua quá trình biên dịch.

2.1.3 Công nghệ xây dựng Backend

2.1.3.1 Giới thiệu về ngôn ngữ lập trình Java

- Java là một ngôn ngữ lập trình hướng đối tượng, đa nền tảng và mạnh mẽ. Một trong những đặc điểm nổi bật của Java là khả năng "viết một lần, chạy ở mọi nơi" nhờ vào Java Virtual Machine (JVM), cho phép các ứng dụng Java có thể chạy trên bất kỳ hệ điều hành nào mà không cần phải thay đổi mã nguồn.
- Java được sử dụng rộng rãi trong việc phát triển ứng dụng web trong môi trường doanh nghiệp. Ngôn ngữ này hỗ trợ tính đa luồng (multithreading), quản lý bộ nhớ tự động qua garbage collection, và có một hệ sinh thái rất phong phú với hàng triệu thư viện và framework.
- Các tính năng chính của Java:
 - Độc lập nền tảng: Java có thể chạy trên bất kỳ hệ điều hành nào với JVM.
 - Hướng đối tượng: Java hỗ trợ các nguyên lý của lập trình hướng đối tượng như kế thừa, đa hình, đóng gói và trừu tượng.
 - Mạnh mẽ và an toàn: Java cung cấp cơ chế xử lý lỗi mạnh mẽ (Exception handling) và có nhiều tính năng bảo mật.
 - Thư viện phong phú: Java có một thư viện API rộng lớn, hỗ trợ rất nhiều công cụ và tính năng sẵn có.
 - Tính đồng thời: Java hỗ trợ lập trình đồng thời (multithreading), giúp tối ưu hiệu suất ứng dụng.

2.1.3.2 Giới thiệu Spring Boot

- **Spring Boot** là một framework phát triển ứng dụng Java được xây dựng trên nền tảng của Spring Framework. Mục tiêu của Spring Boot

là giúp lập trình viên tạo ra các ứng dụng Java đơn giản, nhanh chóng và dễ dàng triển khai mà không cần phải cấu hình phức tạp. Spring Boot làm việc dựa trên nguyên lý "Convention over Configuration" (Cấu hình theo chuẩn thay vì cấu hình thủ công), giúp giảm thiểu lượng công việc cần làm khi cấu hình ứng dụng.

- Spring Boot giúp lập trình viên dễ dàng tạo ra các ứng dụng RESTful, các dịch vụ microservices, hoặc các ứng dụng web. Nó tự động cấu hình các thành phần cần thiết và hỗ trợ việc triển khai ứng dụng trong các môi trường như Cloud, Docker, hoặc Kubernetes.
- Các tính năng chính của Spring Boot:
 - **Auto-configuration:** Spring Boot tự động cấu hình các thành phần của ứng dụng dựa trên các thư viện mà bạn đã thêm vào dự án. Điều này giúp giảm thiểu việc cấu hình thủ công, ví dụ như cấu hình datasource, web server, hoặc các thành phần Spring khác.
 - **Standalone Application:** Spring Boot cho phép bạn xây dựng các ứng dụng độc lập, không cần phải triển khai vào các ứng dụng server như Tomcat hay Jetty. Ứng dụng Spring Boot có thể chạy như một ứng dụng độc lập với một embedded web server
 - **Production Ready:** Spring Boot cung cấp các tính năng hữu ích cho các ứng dụng trong môi trường sản xuất, bao gồm hỗ trợ giám sát, quản lý cấu hình, theo dõi hiệu suất và các API REST để dễ dàng kiểm tra ứng dụng.
 - **Spring Boot Starter:** Các starters là các bộ sưu tập cấu hình và thư viện để hỗ trợ nhanh chóng các tính năng phổ biến, ví dụ như springboot-starter-web cho phát triển ứng dụng web hoặc spring-boot-starterdata-jpa cho truy cập cơ sở dữ liệu.
 - **Spring Boot CLI (Command Line Interface):** Spring Boot cung cấp một CLI để phát triển ứng dụng Java nhanh chóng từ dòng lệnh mà không cần phải sử dụng IDE.

2.1.3.3 Giới thiệu về PostgreSQL

PostgreSQL (hay còn gọi là Postgres) là hệ quản trị cơ sở dữ liệu quan hệ đối tượng (Object-Relational Database Management System - ORDBMS)

mã nguồn mở tiên tiến nhất hiện nay. Nó được phát triển dựa trên dự án POSTGRES tại Đại học California, Berkeley.

Các đặc điểm nổi bật:

- **Tuân thủ chuẩn ACID:** Đảm bảo tính nguyên tử (Atomicity), nhất quán (Consistency), cô lập (Isolation) và bền vững (Durability) của giao dịch, giúp dữ liệu luôn an toàn ngay cả khi hệ thống gặp sự cố.
- **Hỗ trợ đa dạng kiểu dữ liệu:** Ngoài các kiểu dữ liệu quan hệ truyền thống, PostgreSQL còn hỗ trợ mạnh mẽ các kiểu dữ liệu phi cấu trúc như JSON/JSONB (giúp nó hoạt động như một NoSQL) và dữ liệu hình học/địa lý (thông qua PostGIS).
- **Cơ chế MVCC (Multi-Version Concurrency Control):** Cho phép quản lý đồng thời nhiều phiên làm việc mà không xảy ra xung đột khóa (locking conflicts), giúp tăng hiệu suất hệ thống khi có nhiều người dùng truy cập cùng lúc.
- **Khả năng mở rộng (Extensibility):** Người dùng có thể tự định nghĩa các kiểu dữ liệu, hàm, toán tử và ngôn ngữ kịch bản (như PL/pgSQL, Python, Perl)

2.1.3.4 Giới thiệu về Redis Stream

- **Khái niệm Redis Streams** Redis Streams là cấu trúc dữ liệu lưu trữ một chuỗi các sự kiện (events) theo thứ tự thời gian. Mỗi mục (entry) trong Stream là một tập hợp các cặp key-value, được định danh bằng một ID duy nhất (thường là timestamp).
- **Các thành phần chính:**
 - **Entry (Bản ghi):** Đơn vị dữ liệu nhỏ nhất trong Stream. Mỗi Entry có một ID độc nhất (mặc định được tạo dựa trên thời gian: `<millisecondsTime>-<sequenceNumber>`) và chứa nội dung dữ liệu dưới dạng hash (field-value).
 - **Producer (Nhà sản xuất):** Các ứng dụng hoặc dịch vụ đẩy dữ liệu vào Stream bằng lệnh `XADD`.
 - **Consumer (Người tiêu dùng):** Các ứng dụng đọc dữ liệu từ Stream để xử lý.
 - **Consumer Groups (Nhóm tiêu dùng):** Đây là tính năng quan trọng nhất của Redis Streams. Nó cho phép một nhóm các Consumer cùng chia sẻ nhiệm vụ đọc một Stream. Redis đảm bảo mỗi tin nhắn chỉ

được gửi đến một Consumer duy nhất trong nhóm, giúp cân bằng tải (load balancing) và tăng tốc độ xử lý.

- **Cơ chế hoạt động và độ tin cậy:**

- **Persistence (Lưu trữ bền vững):** Dữ liệu trong Stream được lưu trữ trong bộ nhớ (và đĩa cứng tùy cấu hình Redis), không bị mất đi ngay cả khi Consumer chưa kịp đọc (khác với Pub/Sub).
- **Acknowledgment (Cơ chế xác nhận):** Khi hoạt động trong Consumer Group, Redis theo dõi tin nhắn nào đang được xử lý (Pending). Consumer cần gửi tín hiệu xác nhận (**XACK**) sau khi xử lý xong để Redis đánh dấu hoàn thành. Điều này đảm bảo không bị mất dữ liệu nếu Consumer gặp sự cố giữa chừng.

2.2 Kỹ thuật RAG và Xử lý ngôn ngữ tự nhiên để làm chatbot

2.2.1 Giới thiệu về LLM (Large Language Models)

Large Language Models (LLM) là các mô hình trí tuệ nhân tạo được huấn luyện trên một lượng dữ liệu văn bản khổng lồ nhằm học cách hiểu, phân tích và sinh ngôn ngữ tự nhiên tương tự con người. LLM có khả năng xử lý ngữ nghĩa, ngữ cảnh và sinh phản hồi thông minh dựa trên yêu cầu của người dùng. Các mô hình tiêu biểu hiện nay gồm GPT (OpenAI), Llama (Meta) và Gemini (Google).

LLM mang lại nhiều lợi ích khi ứng dụng vào chatbot:

- Khả năng hội thoại tự nhiên: trả lời câu hỏi theo ngữ cảnh, mạch lạc và gần giống con người.
- Hiểu nhiều ngôn ngữ: trong đó một số mô hình hỗ trợ tiếng Việt tốt.
- Mở rộng đa nhiệm: trả lời câu hỏi, tóm tắt văn bản, viết nội dung, phân tích dữ liệu, hỗ trợ học tập.
- Giảm chi phí xây dựng mô hình: không cần phải huấn luyện từ đầu, chỉ cần tinh chỉnh hoặc kết hợp với RAG để dùng cho bài toán chuyên ngành.

2.2.2 Giới thiệu về kỹ thuật RAG



Hình 2-1 : Giới thiệu RAG

Trong các hệ thống Chatbot AI hiện đại, yêu cầu về độ chính xác và khả năng tùy biến theo dữ liệu nội bộ ngày càng cao. Các mô hình ngôn ngữ lớn (Large Language Model – LLM) tuy có khả năng sinh văn bản mạnh mẽ, nhưng kiến thức của chúng bị giới hạn trong phạm vi dữ liệu huấn luyện và có thể bị lỗi “hallucination” (bịa thông tin).

Để khắc phục nhược điểm này, kỹ thuật RAG – Retrieval-Augmented Generation được áp dụng nhằm kết hợp khả năng truy xuất dữ liệu thật với khả năng sinh của LLM

2.2.3 Tổng quan về RAG

RAG (Retrieval-Augmented Generation) là kỹ thuật cho phép mô hình ngôn ngữ truy xuất các tài liệu liên quan từ kho dữ liệu bên ngoài và sử dụng những thông tin đó làm ngữ cảnh để sinh câu trả lời.

Khác với cách sinh thuần túy dựa vào mô hình, RAG đảm bảo:

- Câu trả lời chính xác hơn, dựa trên dữ liệu thật.
- Dễ dàng tùy biến theo tài liệu doanh nghiệp.
- Giảm thiểu hiện tượng suy luận sai.
- Không cần phải huấn luyện lại mô hình mỗi khi cập nhật dữ liệu.

Lý do lựa chọn kỹ thuật RAG cho hệ thống Chatbot AI phục vụ sinh viên:

Việc xây dựng một hệ thống Chatbot hỗ trợ sinh viên đòi hỏi mô hình phải cung cấp thông tin chính xác, được cập nhật thường xuyên và phù hợp với dữ liệu nội bộ của trường. Tuy nhiên, các mô hình ngôn ngữ lớn (LLM) như GPT, Gemini hay LLaMA chỉ được huấn luyện trên dữ liệu công khai, không bao gồm các quy chế đào tạo, thông báo học vụ, lịch thi, quy định tín chỉ hay quy trình hành chính của

từng trường. Điều này dẫn đến hạn chế lớn: nếu dựa hoàn toàn vào LLM, chatbot dễ đưa ra thông tin sai, lỗi thời hoặc không đúng với quy định của PTIT.

Để giải quyết vấn đề này, nhóm lựa chọn kỹ thuật RAG (Retrieval-Augmented Generation). RAG kết hợp khả năng sinh ngôn ngữ mạnh mẽ của LLM với cơ chế truy xuất dữ liệu thật từ một kho tri thức riêng của trường.

2.2.4 Lợi ích của việc sử dụng RAG:

- Đảm bảo tính chính xác của thông tin

Chatbot chỉ trả lời dựa trên tài liệu được truy xuất từ các nguồn chính thống như thông báo trường, văn bản quy chế, hoặc thông tin đã được xác minh từ forum. Điều này giúp giảm thiểu tối đa tình trạng ảo giác “hallucination”.

- Dễ dàng cập nhật dữ liệu mà không cần huấn luyện lại mô hình

Khi trường ban hành thông báo mới hoặc thay đổi quy định, nhóm chỉ cần bổ sung vào Vector Database. Chatbot sẽ tự động truy xuất và trả lời theo dữ liệu mới nhất.

- Tùy biến theo dữ liệu nội bộ của trường

Các LLM không có sẵn kiến thức về quy chế đào tạo và quy trình hành chính riêng của PTIT. RAG cho phép xây dựng một Knowledge Base chuyên biệt, giúp chatbot trả lời đúng với thực tế của sinh viên PTIT.

- Khả năng giải thích và trích dẫn nguồn gốc thông tin

Do câu trả lời được tạo dựa trên các đoạn văn bản truy xuất, chatbot có thể đưa ra câu trả lời “có nguồn gốc”, tăng độ tin cậy.

- RAG chỉ cần gọi API LLM kết hợp với tìm kiếm vector, tiết kiệm đáng kể tài nguyên.

2.2.5 Vector đặc trưng

Vector đặc trưng (Feature Vector) là một dạng biểu diễn số hóa của dữ liệu, trong đó mỗi đối tượng (một đoạn văn, hình ảnh, âm thanh hoặc dữ liệu cảm biến...) được mô tả bằng một dãy số có thứ tự. Mỗi giá trị trong vector biểu thị một đặc trưng (feature) quan trọng của đối tượng đó.

Trong dự án này, vector đặc trưng sẽ được sinh ra từ mô hình ngôn ngữ lớn, cụ thể là: gemini embedding-exp-03-07

2.2.6 Vector Database

Vector Database (Cơ sở dữ liệu vector) là một hệ thống lưu trữ và truy vấn dữ liệu được tối ưu hóa cho các vector nhiều chiều. Đây là loại cơ sở dữ liệu đặc biệt được

thiết kế để xử lý các embedding — tức các biểu diễn dạng vector của văn bản, hình ảnh hoặc âm thanh.

Khác với cơ sở dữ liệu truyền thống (quan hệ hoặc NoSQL) vốn hoạt động dựa trên việc so khớp chính xác các giá trị, Vector Database tập trung vào việc tìm kiếm độ tương đồng ngữ nghĩa giữa các vector. Khi một truy vấn được gửi vào, hệ thống sẽ đo lường khoảng cách hoặc độ tương tự giữa các vector (thường sử dụng cosine similarity, Euclidean distance...) để xác định những điểm dữ liệu gần nhất về mặt ý nghĩa.

Nhờ cơ chế hoạt động này, Vector Database đặc biệt phù hợp trong hệ thống RAG, nơi chatbot cần:

- Hiểu được câu hỏi của người dùng dưới dạng ngữ nghĩa
- Tìm nhanh các đoạn văn có nội dung gần nhất với câu hỏi
- Truy xuất thông tin chính xác ngay cả khi câu hỏi không trùng từ khóa

Vector Database đóng vai trò như “bộ nhớ” của hệ thống RAG với đặc điểm tìm kiếm theo ngữ nghĩa (Semantic Search)

Thay vì tìm kiếm bằng từ khóa, Vector Database sử dụng các thuật toán đo độ tương đồng giữa các vector (ví dụ cosine similarity) để tìm ra những đoạn văn có ý nghĩa gần nhất với câu hỏi của người dùng.

Cosine similarity

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Trong đó:

- $A \cdot B$ là tích vô hướng (dot product)
- $\|A\|$ và $\|B\|$ là độ dài (magnitude) của hai vector

Kết quả nằm trong khoảng:

- **1** → hai vector **trùng hướng** (rất giống nhau)
- **0** → **vuông góc**, không liên quan
- **-1** → **ngược hướng**, hoàn toàn khác nhau

Khi người dùng hỏi trong chatbot, câu hỏi của người dùng sẽ được nhúng thành vector (trong hệ thống là thông qua việc gọi API từ model google), kết quả trả về là một vector embedding đại diện cho câu hỏi của người dùng. Sau khi có vector

embedding của câu hỏi thì có thể áp dụng Cosine similarity, để tìm ra k vector có độ tương đồng cao nhất trong vector database để làm tài liệu cho AI trả lời người dùng.



Hình 2-2 : Giới thiệu Pinecone

Pinecone là một **Vector Database** được cung cấp dưới dạng dịch vụ (Vector DBaaS) chuyên dụng cho các ứng dụng AI như tìm kiếm ngữ nghĩa, RAG, phân cụm và gợi ý thông minh. Pinecone cho phép lưu trữ và truy vấn hàng triệu vector embedding với tốc độ rất nhanh và độ chính xác cao.

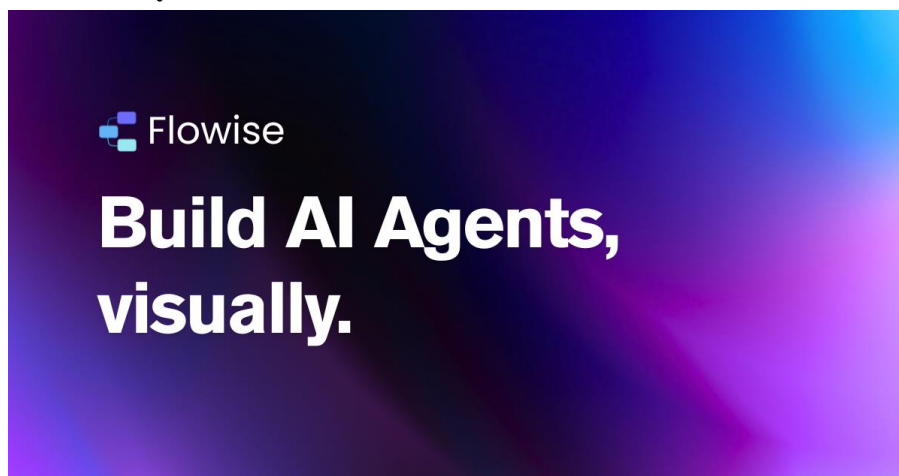
Những ưu điểm tiêu biểu:

- **Hiệu năng truy vấn cao:** Tối ưu cho tìm kiếm vector gần đúng (ANN) với độ trễ thấp, phù hợp cho chatbot thời gian thực.
- **Quản lý dữ liệu đơn giản:** Không cần tự vận hành server; Pinecone lo toàn bộ scaling, phân mảnh và tối ưu hóa.
- **Tích hợp dễ dàng:** Hỗ trợ API đơn giản, dùng tốt với LangChain, LlamaIndex, Flowise và các mô hình embedding phổ biến như Gemini, OpenAI, Cohere.

Chính vì những đặc điểm như vậy nhóm đã quyết định sử dụng để làm nơi lưu trữ vector

2.2.7 Triển khai RAG

2.2.7.1 Giới thiệu về Flowise



Hình 2-3 : Giới thiệu Flowise

Flowise là một nền tảng open-source cho phép xây dựng các ứng dụng AI, đặc biệt là các hệ thống dựa trên kiến trúc RAG (Retrieval-Augmented Generation), thông qua giao diện kéo-thả trực quan. Flowise được phát triển nhằm hỗ trợ người dùng triển khai các chatbot AI, workflow xử lý ngôn ngữ tự nhiên, cũng như kết nối với nhiều mô hình LLM và vector database một cách dễ dàng, không đòi hỏi quá nhiều kiến thức lập trình chuyên sâu.

Một ưu điểm nổi bật của Flowise so với việc lập trình RAG thủ công là khả năng tích hợp rộng với nhiều dịch vụ AI phổ biến hiện nay như OpenAI, Google AI, Hugging Face, Pinecone, Weaviate... cùng với việc hỗ trợ export API trực tiếp. Điều này giúp Flowise trở thành công cụ phù hợp cho các nhóm phát triển AI muốn rút ngắn thời gian xây dựng thử nghiệm, tối ưu hoá quy trình vận hành và dễ dàng tích hợp vào sản phẩm thực tế.

Trong đồ án này, Flowise đóng vai trò là nền tảng trung gian để triển khai kiến trúc RAG, giúp nhóm thiết kế chatbot AI dành cho sinh viên một cách nhanh chóng, trực quan và hiệu quả.

2.2.7.2 Thu thập dữ liệu từ website trường

- Công cụ crawl dữ liệu dùng thư viện Jsoup :

Sử dụng thư viện Jsoup để lấy nội dung các bài viết và tiêu đề trên các trang web công khai của nhà trường như :

<https://ptit.edu.vn/tin-tuc-su-kien/thong-bao>

<https://daotao.ptit.edu.vn/tin-tuc/tin-tuc-chung/>

<https://giaovu.ptit.edu.vn/category/thong-bao/>

Đây là các trang web tĩnh do đó có thể đọc trực tiếp nội dung từ các thẻ html như thẻ a, h1,...

- Cơ chế lập lịch (Scheduler) để tự động lấy tin mới từ trang trường.

Cài đặt cơ chế lập lịch, tự động chạy mỗi 2h sáng hằng ngày để quét lấy các thông tin mới (nếu có) để cập nhật trên web hỗ trợ để sinh viên nhận được tin tức tổng hợp từ nhiều nguồn một cách nhanh chóng nhất.

2.2.7.3 Thu thập dữ liệu từ forum và file gửi kèm thông báo tới sinh viên

Trong dự án, nhóm triển khai một nền tảng forum trao đổi dành cho sinh viên, nơi người dùng có thể đăng bài viết để đặt câu hỏi, chia sẻ thắc mắc hoặc trao đổi kinh nghiệm học tập, đặc biệt hữu ích đối với các sinh viên năm nhất. Các bài viết trên forum sau khi được cộng đồng đánh giá cao và được ban quản trị xác thực sẽ được xem như các nguồn tri thức đáng tin cậy. Những bài viết này được thu thập, chuẩn hoá và đưa vào kho dữ liệu để sử dụng làm nguồn kiến thức cho hệ thống chatbot AI.

Bên cạnh đó, trong mục thông báo gửi đến sinh viên, nhiều nội dung đi kèm các tệp tài liệu như PDF, Word hoặc hình ảnh chứa thông tin chi tiết về quy định, hướng dẫn, kế hoạch học tập... Các tệp đính kèm này cũng được hệ thống quản lý và thu thập lại. Nhóm tiến hành trích xuất nội dung, làm sạch dữ liệu và đưa chúng vào cơ sở tri thức của chatbot nhằm đảm bảo chatbot có thể cung cấp câu trả lời chính xác dựa trên các tài liệu chính thức của nhà trường.

CHƯƠNG III: Phân tích và Thiết kế hệ thống (System Analysis & Design)

3.1 Phân tích yêu cầu (Functional Requirements):

3.1.1 Xác định các tác nhân của hệ thống :

3.1.1.1 Hệ thống chính:

User	- Xem thông báo - Tìm kiếm tài liệu - Tính toán CPA - Tham gia forum
Admin	- Quản lý thông báo - Quản lý tài liệu - Quản lý môn học - Quản lý thông tin các khoa - Quản lý category - Quản lý tất cả topic trong forum - Quản lý tất cả post trong forum - Quản lý thông tin người dùng - Trích xuất file dữ liệu

Bảng 3-1: Tác nhân hệ thống chính

3.1.1.2 Hệ thống nghiệp vụ:

Quản lý Topic	- Duyệt bài đăng - Duyệt thêm thành viên - Mời thành viên mới - Xử lý bài đăng bị report - Xử lý comment bị report
Thành viên Topic	- Xem bài đăng - Tạo bài đăng mới - Bình luận - Reaction các bài đăng và bình luận - Report comment hay bình luận

*Bảng 3-2: Tác nhân hệ thống nghiệp vụ***3.1.2 Xác định các yêu cầu chức năng**

Chức năng của từng tác nhân được mô tả bằng ngôn ngữ tự nhiên như sau :

- Admin quản lý thông báo: Admin đăng nhập -> Chọn quản lý thông báo -> Hệ thống hiển thị danh sách các thông báo -> Chọn tạo thông báo mới -> Hệ thống hiển thị danh sách các thông báo đã cập nhật -> Gửi một thông báo cho người dùng liên quan -> Cập nhật lại trạng thái các thông báo (danh sách đã gửi và danh sách chưa gửi) -> Chọn xoá một thông báo -> Xác nhận hành động -> Cập nhật lại danh sách các thông báo.
- Admin quản lý môn học: Admin đăng nhập -> Chọn quản lý môn học -> Hệ thống hiển thị danh sách các môn học -> Chọn thêm môn học mới -> Hiển thị danh sách môn học đã cập nhật
- Admin quản lý kỳ học: Admin đăng nhập -> Chọn quản lý kỳ học -> Hệ thống hiển thị danh sách các kỳ học -> Chọn thêm kỳ học mới -> Hiển thị danh sách kỳ học đã cập nhật
- Admin quản lý khoa: Admin đăng nhập -> Chọn quản lý khoa -> Hệ thống hiển thị danh sách các khoa -> Chọn thêm khoa mới -> Hiển thị danh sách khoa đã cập nhật
- Admin quản lý tham chiếu môn học: Admin đăng nhập -> Chọn quản lý khoa -> Hệ thống hiển thị danh sách các khoa -> Chọn thêm khoa mới -> Hiển thị danh sách khoa đã cập nhật
- Admin quản lý Category: Admin đăng nhập -> Chọn quản lý Category -> Hệ thống hiển thị danh sách các Category -> Chọn thêm Category mới -> Hiển thị danh sách Category đã cập nhật
- Người dùng xem thông báo: User đăng nhập -> Chọn xem thông báo -> Hệ thống hiển thị danh sách thông báo -> Chọn một thông báo -> Hệ thống hiển thị chi tiết thông báo.

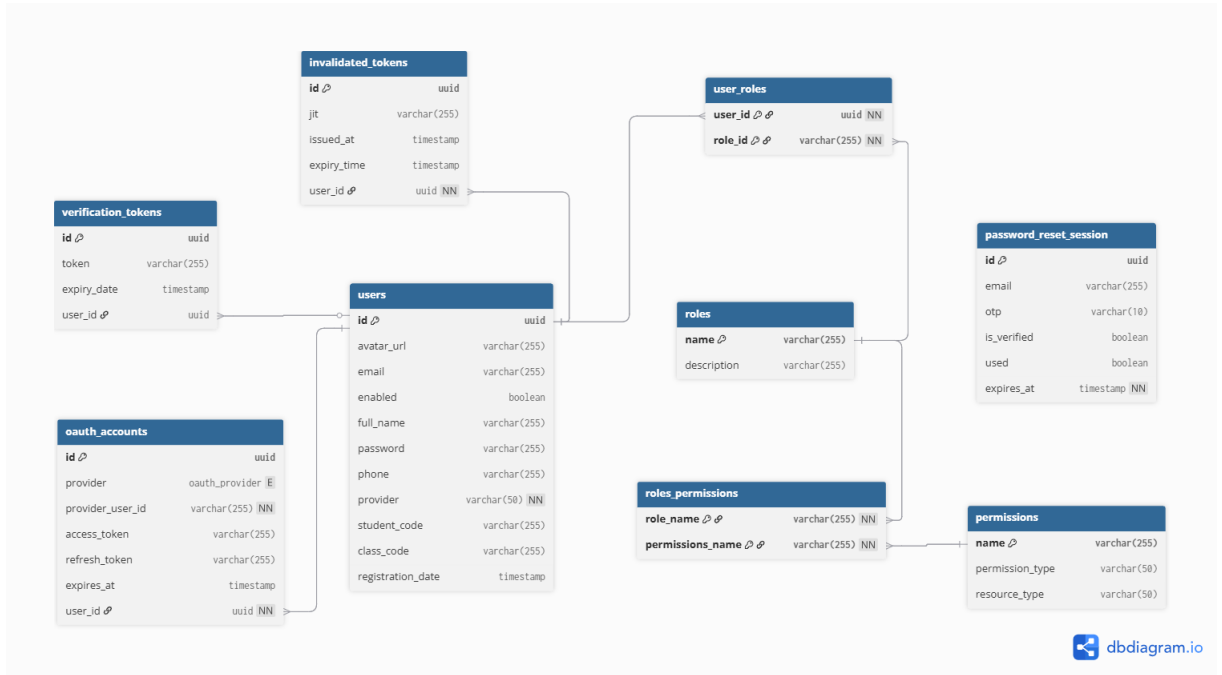
- Người dùng tính điểm : User đăng nhập -> Chọn chức năng tính điểm -> Hệ thống hiển thị giao diện tính điểm -> Nếu chưa từng tạo, chọn khởi tạo -> Hệ thống hiển thị các môn thuộc kỳ học đầu tiên của sinh viên đó (dựa trên mã sinh viên) -> Chọn thêm kỳ -> Hiển thị thêm kỳ -> Nhập điểm -> Hiển thị điểm CPA, GPA đã tính.
- Người dùng xem bài đăng: User đăng nhập -> Chọn forum -> Hệ thống hiển thị danh sách các Category -> Chọn Category -> Hiển thị danh sách các Topic -> Chọn một topic (nếu public) -> Hiển thị danh sách các bài đăng trong topic -> Chọn tạo bài đăng mới -> Hệ thống hiển thị giao diện tạo bài đăng -> Người dùng xác nhận tạo -> Hệ thống hiển thị danh sách các bài đăng trong topic.
- Quản lý topic duyệt yêu cầu tham gia : Quản lý topic đăng nhập -> Chọn forum -> Hệ thống hiển thị danh sách các Category -> Chọn Category -> Hiển thị danh sách các Topic -> Chọn một topic bản thân tham gia quản lý -> Hiển thị danh sách các bài đăng -> Chọn xem các yêu cầu tham gia -> Hệ thống hiển thị các yêu cầu tham gia -> Người dùng duyệt các yêu cầu này.

3.2 Biểu đồ Usecase tổng quát:

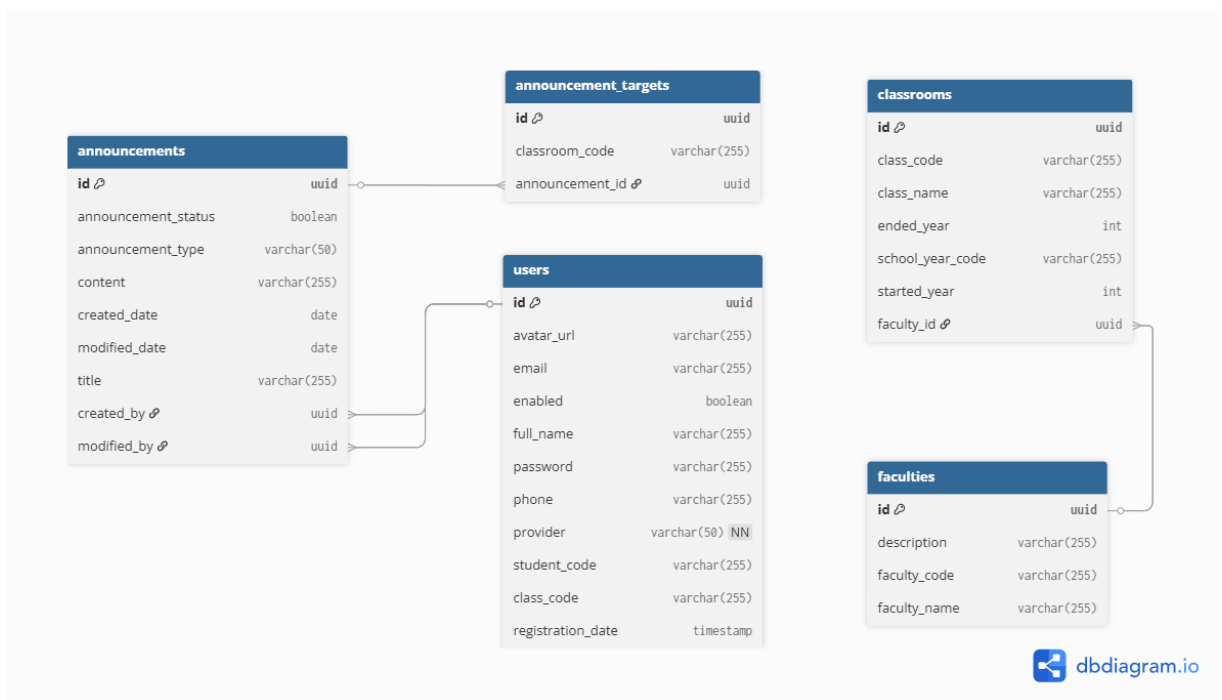


Hình 3-1: Biểu đồ Usecase tổng quát

3.3 Thiết kế Cơ sở dữ liệu (ERD):

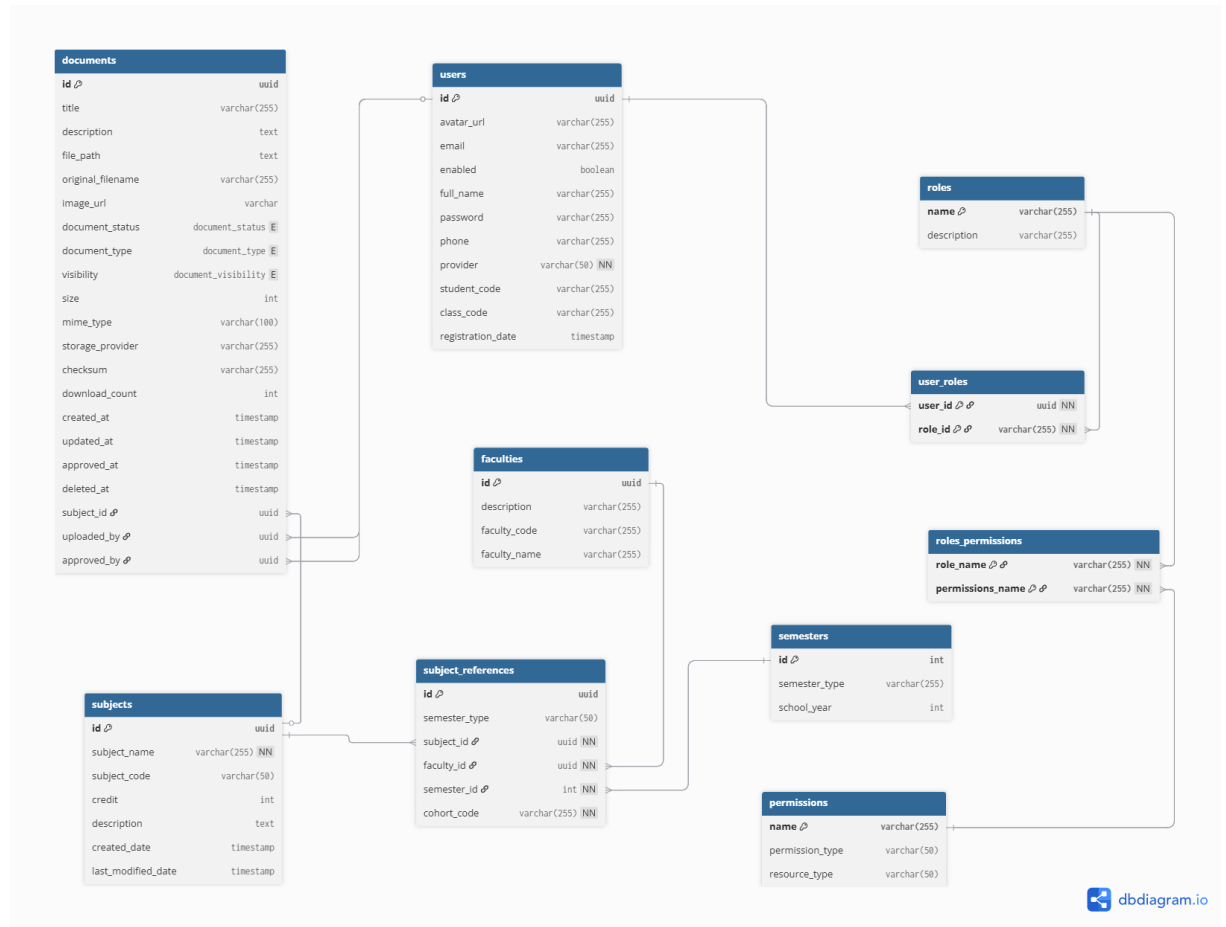


Hình 3-2: Các bảng cho xác thực, phân quyền

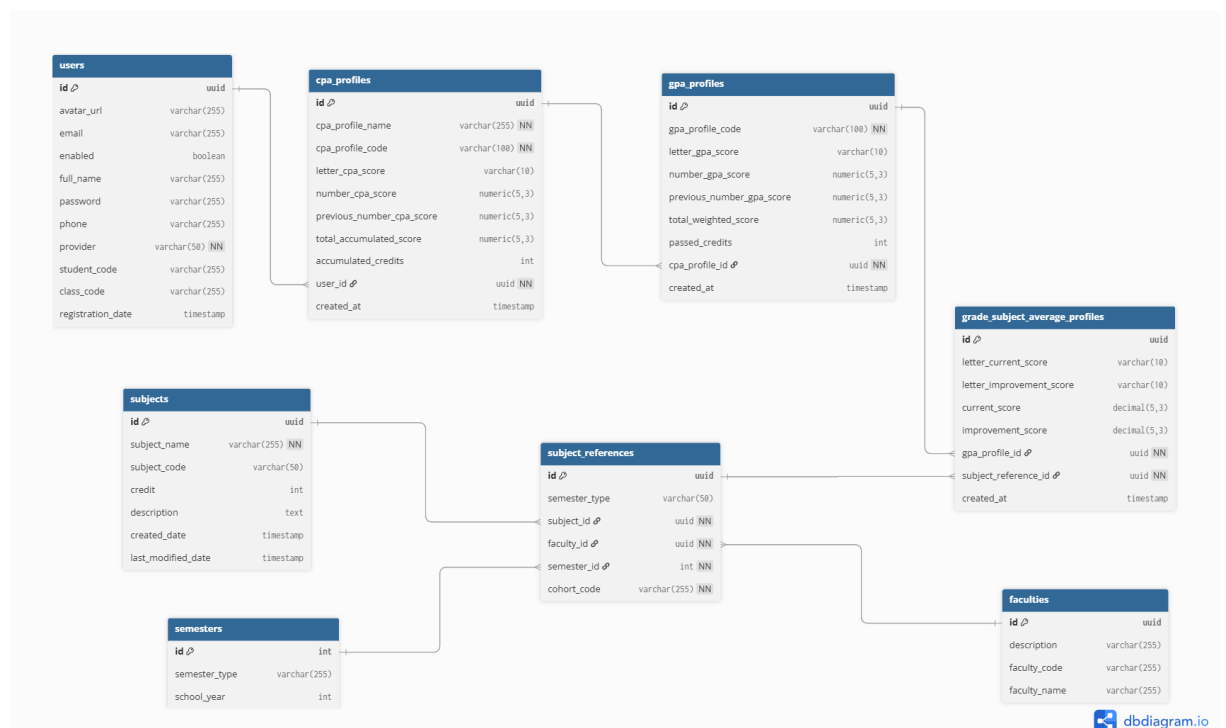


Hình 3-3: Các bảng cho chức năng thông báo

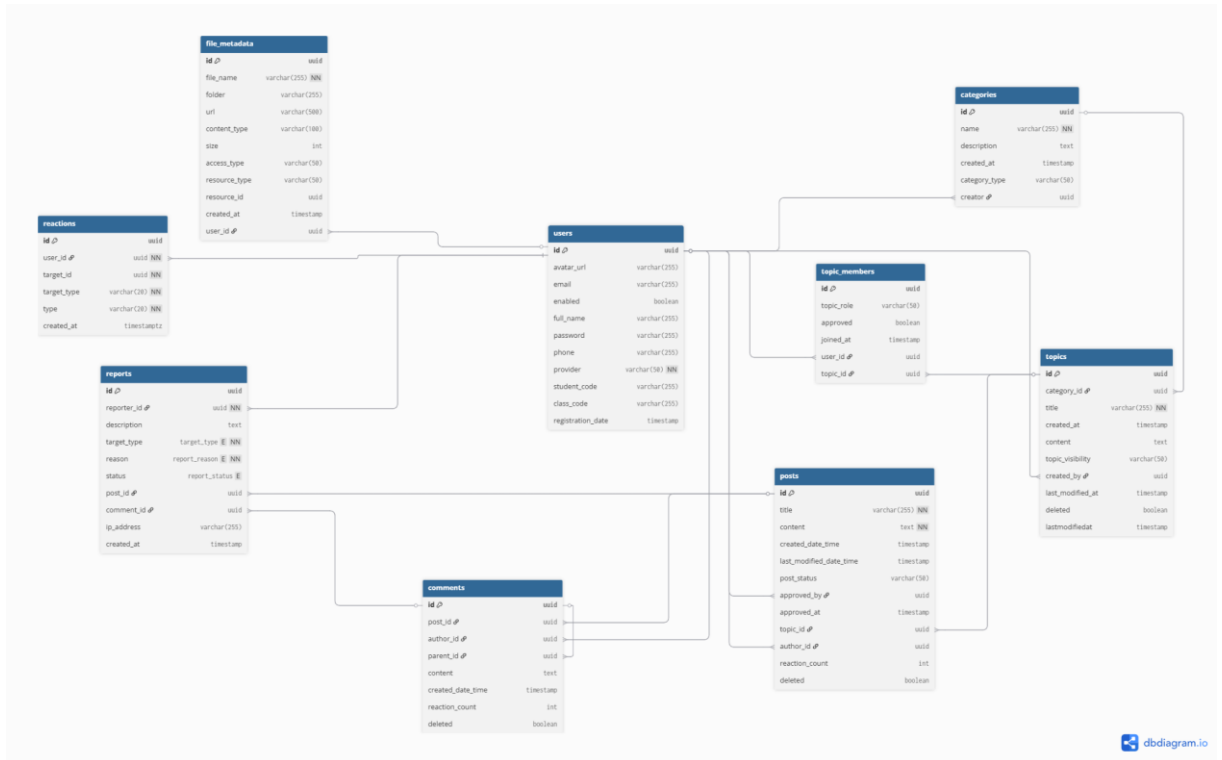
Các bảng cho chức năng thư viện



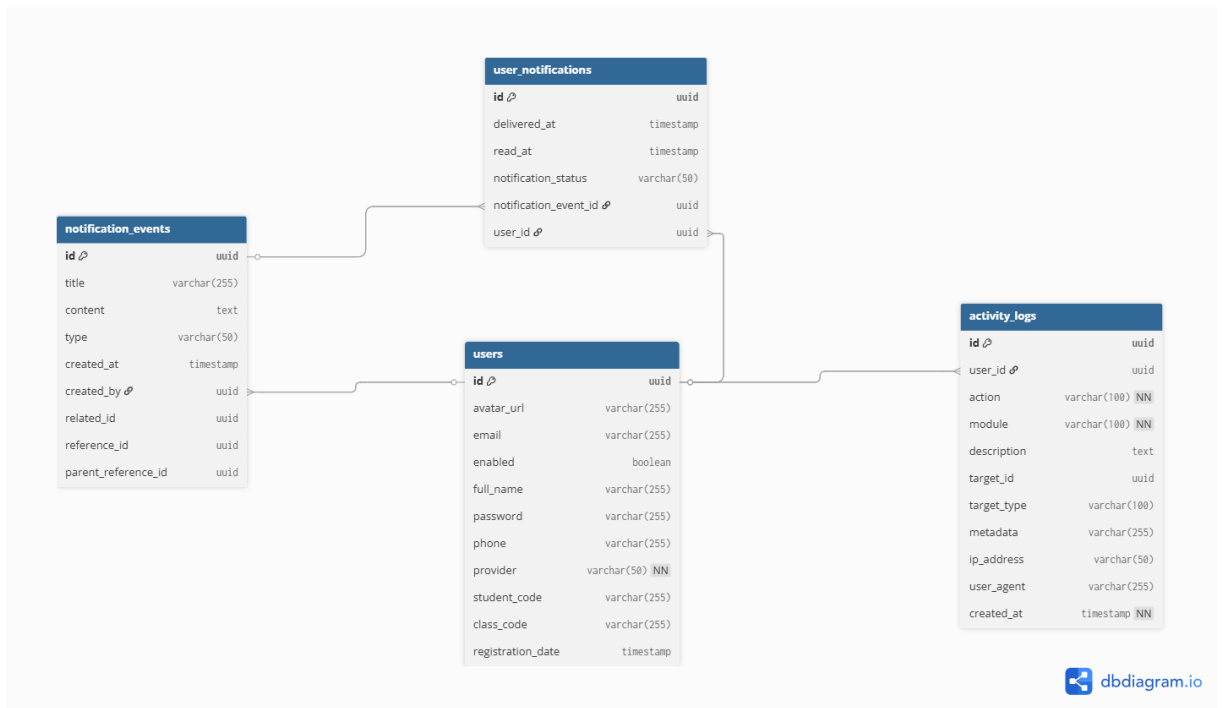
Hình 3-4: Các bảng cho chức năng thư viện



Hình 3-5: Các bảng cho chức năng tính điểm



Hình 3-6: Các bảng cho chức năng forum



Hình 3-7: Các bảng cho chức năng quản lý log, thông báo

3.4 Thiết kế Kiến trúc Hệ thống :

3.4.1 Kiến trúc Frontend

Hệ thống Frontend không được xây dựng theo mô hình MVC truyền thống hay gom theo loại file (folder **components**, **hooks** chung chung) mà áp dụng kiến trúc **Modular Feature-based Architecture** trên nền tảng **Next.js App Router**

Kiến trúc này được thiết kế để giải quyết bài toán về khả năng mở rộng và bảo trì khi dự án phình to với nhiều nghiệp vụ phức tạp như **Forum**, **Chatbot** và **Quản lý sinh viên**

3.4.1.1 Cấu trúc thư mục hướng nghiệp vụ (Domain-Driven Directory Structure)

Thay vì tổ chức code theo loại file (Tech-driven), hệ thống tổ chức theo tính năng(Domain-driven). Cụ thể cấu trúc **src/** được chia thành các tầng rõ ràng theo hình dưới đây:

Lý do chọn kiến trúc:

- Tính cục bộ: Code liên quan tới chức năng “Đăng bài” (Form, Validate, API) nằm gọn trong **features/post**. Khi cần sửa các chức năng đăng bài, Dev chỉ cần vào trong folder này không cần tìm rải rác khắp dự án
- Tính cô lập: các feature hạn chế phụ thuộc chéo. Nếu xóa tính năng “Forum” chỉ cần xóa thư mục **features/post** và **features/forum** mà không làm crash trang “Dashboard”
- Tính mở rộng: Dễ dàng chia việc cho team. Mỗi người làm một chức năng mà không conflict code

3.4.1.2 Chiến lược Rendering: Hybrid Rendering(Server + Client)

Sử dụng **Next.js App Router** cho phép hệ thống kết hợp linh hoạt giữa Server Components (RSC) và Client Components

- Server Components : được sử dụng cho các trang tĩnh hoặc ít tương tác như trang chủ, Chi tiết thông báo
 - Lợi ích: giảm bundle size gửi xuống client, tăng tốc độ tải ban đầu, tối ưu SEO
- Client Components : Được sử dụng cho các module tương tác cao như Forum, Chatbot, Real-time, Notification
 - Lợi ích: tận dụng sức mạnh của trình duyệt để xử lý sự kiện phức tạp, quản lý state (React Query) và cập nhật giao diện tức thì (Optimistic Ui) mà không cần reload trang

3.4.1.3 Hệ thống UI: Headless & Utility-First

Hệ thống không sử dụng các thư viện UI “cứng” như Bootstraps hay AntD mà xây dựng dựa trên stack hiện đại:

- Shadcn/UI (Radix UI + Tailwind CSS):
 - Lý do: Shadcn không phải là thư viện đóng gói sẵn mà là các component copy-paste. Điều này cho phép toàn quyền kiểm soát code.
 - Accessibility: Đảm bảo tiêu chuẩn tiếp cận (hỗ trợ phím điều hướng, screen reader)
- Tailwind CSS:
 - Giúp styling nhanh chóng ngay trong file tsx, giảm thiểu việc chuyển đổi ngữ cảnh giữa file logic và file style
 - Dễ dàng truy cập thiết lập Design System đồng bộ thông qua [tailwind.config.ts](#)

3.4.1.4 Lớp giao tiếp API và quản lý State Server

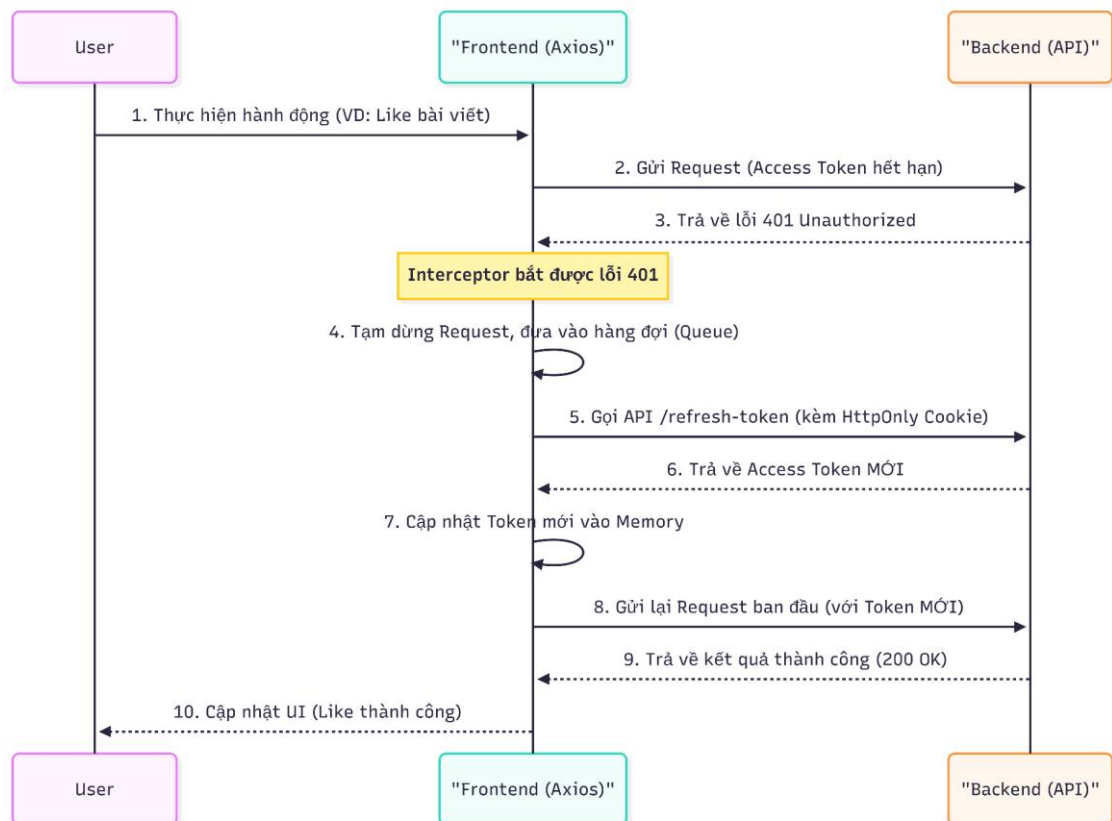
Để xử lý bài toán bất đồng bộ giữa **Frontend** và **Backend**, hệ thống sử dụng mô hình 3 lớp tại phía Client:

- API Client Layer:
 - Sử dụng Axios Interceptor để tự động hóa việc đính kèm Access Token vào header mỗi lần request
 - Cơ chế Auto-refresh Token: Tự động bắt lỗi **401 Unauthorized**, gọi API làm mới token và thực hiện lại request ban đầu một cách trong suốt với người dùng
- Data Fetching Layer
 - Thay thế hoàn toàn useEffect để gọi API
 - Quản lý **Server State**(Caching, deduplication, revalidation). Giúp giao diện Forum phản hồi tức thì cả khi mạng chậm nhờ cơ chế *Stale-while-revalidate*
- Form Management Layer
 - Sử dụng Zod Schema để định nghĩa cấu trúc dữ liệu và validate ngay tại client
 - Schema này được đồng bộ hóa chặt chẽ với DTO của Backend để đảm bảo tính toàn vẹn của dữ liệu khi gửi request

3.4.2 Thiết kế Bảo mật

3.4.2.1 Hệ thống áp dụng chiến lược bảo mật Defense in Depth (Phòng thủ chiều sâu):

Thiết lập nhiều lớp rào chắn từ phía Client (Frontend) đến Server (Backend). Thay vì phụ thuộc vào các giải pháp xác thực có sẵn (như NextAuth) vốn khó tùy biến sâu với Spring Boot, nhóm phát triển tự triển khai kiến trúc bảo mật dựa trên chuẩn JWT (JSON Web Token) kết hợp với Dual-Token Architecture



Hình 3-8: Quy trình xử lý JWT

3.4.2.2 Mô hình bảo mật được thiết kế gồm 3 lớp chính:

1. Kiến trúc Xác thực Dual-Token

Để giải quyết bài toán cân bằng giữa bảo mật (Security) và trải nghiệm người dùng (UX), hệ thống không lưu trữ Token trong LocalStorage/SessionStorage (nơi dễ bị tấn công XSS). Thay vào đó, cơ chế **Dual-Token** được áp dụng như sau:

- Access Token
 - **Vai trò:** Dùng để xác thực các request API thông thường.
 - **Lưu trữ:** Chỉ lưu trong **Bộ nhớ (In-Memory Variable)** của ứng dụng React thông qua Auth Context.

- **Cơ chế:** Token này có thời gian sống ngắn (ví dụ: 15 phút). Vì chỉ nằm trong RAM, nếu hacker thực hiện tấn công XSS (Cross-Site Scripting), họ không thể đọc được token này từ ổ cứng hay storage của trình duyệt. Token sẽ tự động mất khi người dùng đóng tab.
- Refresh Token
 - **Vai trò:** Dùng để cấp lại Access Token mới mà không cần người dùng đăng nhập lại.
 - **Lưu trữ:** Được Backend set vào **HttpOnly, Secure, SameSite Cookie**.
 - **Cơ chế:** JavaScript phía Client hoàn toàn không thể đọc hoặc can thiệp vào Cookie này, giúp ngăn chặn triệt để nguy cơ bị đánh cắp phiên đăng nhập thông qua mã độc.

2. Cơ chế Silent Refresh & Axios Interceptors

Để đảm bảo trải nghiệm người dùng liền mạch (Seamless UX), hệ thống xử lý việc làm mới token một cách tự động và trong suốt (Transparent) thông qua kỹ thuật Intercepting Request/Response:

- Request Interceptor: Tự động đính kèm Access Token hiện có vào Header Authorization: Bearer <token> của mọi request gửi đi.
- Response Interceptor: Hệ thống lắng nghe toàn bộ phản hồi từ Backend.
 - Khi phát hiện lỗi **401 Unauthorized** (dấu hiệu Access Token hết hạn), hệ thống không đăng xuất người dùng ngay lập tức.
 - **Queueing:** Tạm dừng các request bị lỗi và đưa vào hàng đợi.
 - **Refreshing:** Thực hiện gọi API **/api/auth/refresh** (Cookie Refresh Token sẽ tự động được gửi kèm request này bởi trình duyệt) để lấy Access Token mới.
 - **Retrying:** Sau khi có token mới, hệ thống tự động thực thi lại các request trong hàng đợi với token mới.
 - **Kết quả:** Người dùng vẫn tiếp tục thao tác mà không hề biết phiên làm việc đã được gia hạn ngầm

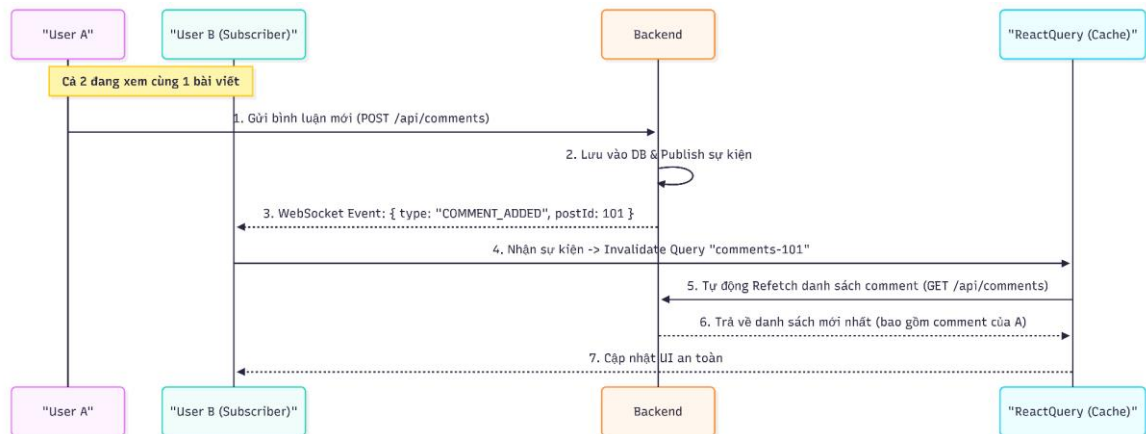
3. Bảo vệ Route và Dữ liệu đầu vào

- Next.js Edge Middleware
 - Hệ thống sử dụng Middleware chạy ở tầng Edge (Server-side) để kiểm soát quyền truy cập trước khi request đến được Page Component.
 - Ngăn chặn hiện tượng "FOUC" (Flash of Unauthenticated Content) - giao diện bị nháy nội dung bảo mật trước khi redirect.
 - Logic điều hướng: Người dùng chưa có phiên làm việc (Cookie) sẽ bị chặn ngay lập tức khi cố truy cập các trang /forum, /profile
- API Proxying
 - Các request nhạy cảm (Đăng nhập, Refresh Token) không được gọi trực tiếp từ Browser sang Spring Boot Backend để tránh lộ cấu trúc API nội bộ.
 - Hệ thống sử dụng **Next.js API Routes** làm lớp Proxy trung gian, giúp ẩn địa chỉ thực của Backend Server và giải quyết triệt để vấn đề CORS (Cross-Origin Resource Sharing).
- Input Validation & Sanitization

- Sử dụng thư viện **Zod** để định nghĩa Schema validation chặt chẽ ngay tại Client.
- Mọi dữ liệu (Form đăng ký, Bài đăng Forum) đều phải đi qua lớp kiểm tra định dạng (Email, Strong Password, độ dài content) trước khi được gửi đi, giúp giảm tải Bad Request cho Server và ngăn chặn các nỗ lực tiêm mã độc cơ bản.

3.4.3 Kiến trúc Thời gian thực

Để giải quyết bài toán tương tác tức thì (Instant Interaction) trong các phân hệ Chatbot và Forum (ví dụ: thông báo bình luận mới, phản hồi từng token từ AI), hệ thống loại bỏ hoàn toàn cơ chế **HTTP Polling** truyền thống (gửi request liên tục mỗi n giây) vì gây lãng phí tài nguyên mạng và độ trễ cao. Thay vào đó, nhóm xây dựng kiến trúc **Event-Driven** dựa trên giao thức WebSocket, sử dụng **STOMP** (Simple Text Oriented Messaging Protocol) làm lớp giao vận.



Hình 3-9: Kiến trúc thời gian thực

Kiến trúc này bao gồm 3 thành phần cốt lõi:

1. Mô hình Pub/Sub (Publisher - Subscriber)

Hệ thống thiết lập một kênh giao tiếp hai chiều bền vững (Duplex Channel) giữa Client và Server, hoạt động theo cơ chế Xuất bản - Đăng ký:

- **Subscriber (Frontend Layer):** Ngay khi khởi tạo phiên làm việc, Client sẽ thiết lập kết nối WebSocket và đăng ký (Subscribe) vào các Topic định danh:
 - **/user/queue/notifications:** Kênh riêng tư (Private Channel) dành cho các thông báo cá nhân (kết quả học tập, phản hồi báo cáo vi phạm).
 - **/topic/forum/updates:** Kênh quảng bá (Broadcast Channel) dành cho các sự kiện cộng đồng (bài viết mới nổi, thông báo toàn trường).
 - **/user/queue/chat.stream:** Kênh stream dữ liệu phản hồi từ AI (Token Streaming).
- **Publisher (Backend Layer):** Khi có sự kiện nghiệp vụ phát sinh (Event Trigger), Backend sẽ đẩy một gói tin (Payload) vào Message Broker. Broker chịu trách

nhiệm phân phối gói tin này xuống đúng Client đang lắng nghe topic tương ứng ngay lập tức (Real-time Push).

2. Chiến lược Quản lý Kết nối

Việc duy trì kết nối WebSocket trong môi trường Web (nơi mạng không ổn định hoặc người dùng chuyển tab) là thách thức lớn. Hệ thống giải quyết bằng lớp quản lý kết nối StompClient (Singleton Pattern) với các cơ chế tự phục hồi:

- **Persistent Connection (Kết nối bền vững):** Kết nối được khởi tạo một lần duy nhất tại tầng cao nhất của ứng dụng (Root Provider) và tái sử dụng cho toàn bộ các màn hình.
- **Heartbeat Mechanism (Nhịp tim):** Client và Server liên tục trao đổi các gói tin ping/pong định kỳ để xác nhận trạng thái kết nối. Nếu không nhận được tín hiệu phản hồi, Client tự động xác định mạng bị gián đoạn.
- **Exponential Backoff Reconnection:** Khi mất kết nối, hệ thống tự động thực hiện thuật toán thử lại theo cấp số nhân (thử lại sau 1s, 2s, 5s...) để khôi phục kết nối mà không làm gián đoạn trải nghiệm người dùng hay gây tấn công DDoS vào chính Server.

3. Chiến lược Đồng bộ Dữ liệu "Event-Triggered Refetch"

Để đảm bảo tính nhất quán dữ liệu (Data Consistency) giữa giao diện và Server, hệ thống áp dụng chiến lược kết hợp giữa **WebSocket** và **TanStack Query** (State Management):

- **Vấn đề:** Nếu Client tự động chèn dữ liệu nhận được từ WebSocket vào danh sách hiển thị (UI), rất dễ xảy ra xung đột dữ liệu (Duplicate, sai thứ tự sắp xếp, thiếu các trường dữ liệu quan hệ).
- **Giải pháp:**
 - **Signal:** WebSocket chỉ đóng vai trò là tín hiệu báo tin (Trigger). Ví dụ: Nhận sự kiện **COMMENT_ADDED**.
 - **Invalidate:** Client nhận tín hiệu và đánh dấu dữ liệu trong Cache là "lỗi thời" (Stale)
 - **Refetch:** React Query tự động kích hoạt một request ngầm (Background Fetch) để lấy dữ liệu mới nhất, chuẩn xác nhất từ Server và cập nhật lại giao diện.

Chương IV: Xây dựng Module Dữ liệu và AI Chatbot

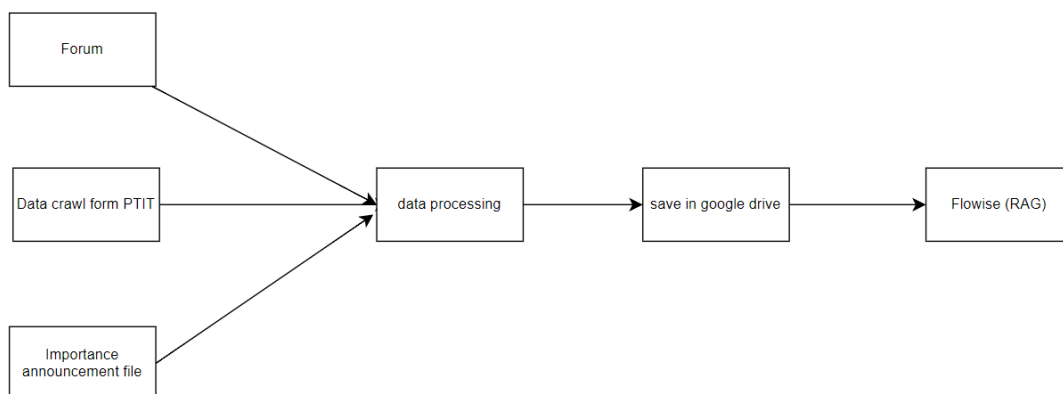
4.1 Giới thiệu

Để xây dựng một Chatbot AI hỗ trợ sinh viên trong việc tra cứu thông tin về quy chế đào tạo, lịch học, thông báo, thủ tục hành chính,... hệ thống cần hai thành phần chính:

- Module thu thập dữ liệu: đảm nhiệm thu thập, làm sạch, tổ chức và chuyển đổi dữ liệu từ nhiều nguồn thành dạng có thể sử dụng cho AI.
- Module AI Chatbot: sử dụng kỹ thuật RAG (Retrieval-Augmented Generation) để kết hợp dữ liệu thực tế và khả năng sinh văn bản của mô hình ngôn ngữ lớn (LLM).

Hai module này kết hợp giúp chatbot trả lời chính xác, cập nhật thường xuyên và tránh sai lệch thông tin.

4.2 Module thu thập dữ liệu



Hình 4-1: Mô hình thu thập dữ liệu

4.2.1 Thu thập dữ liệu từ forum

Forum chứa các bài post hỏi đáp theo các chủ đề. Admin có thể tìm kiếm và lọc các bài post kèm comment đã được xác thực là hữu ích. Các bài post kèm comment hữu ích sẽ được tổng hợp thành 1 file text. Các bài post có định dạng ngăn cách nhau để giúp dễ xử lý và phân chia khi đưa vào RAG

#Hướng dẫn đăng ký môn học Bài viết này hướng dẫn chi tiết cách đăng ký môn học cho sinh viên năm nhất.

Cảm ơn bài viết, rất hữu ích!

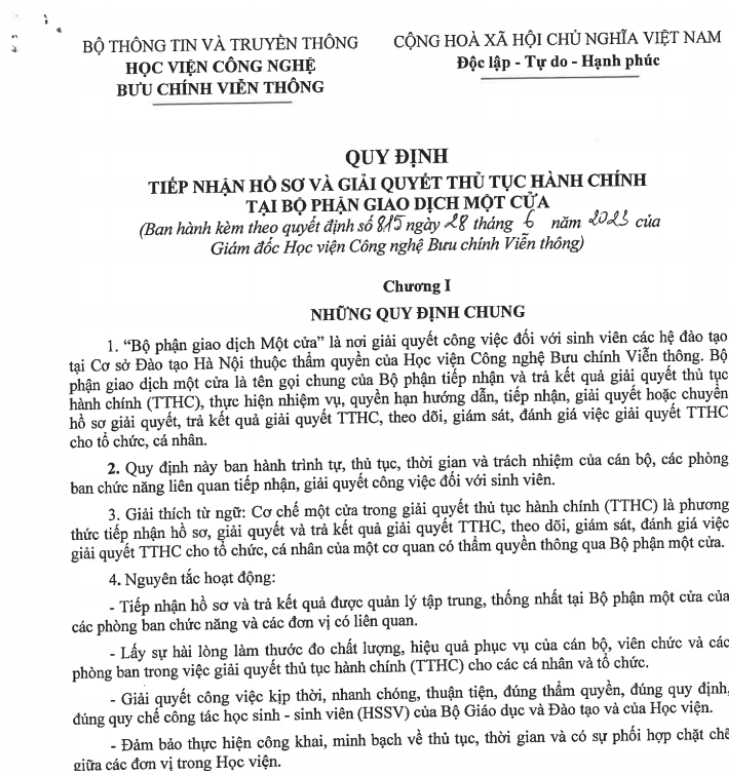
Cho mình hỏi bước 3 là làm gì thế?

#Cách sử dụng thư viện trường Hướng dẫn tra cứu tài liệu và mượn sách trong thư viện.

Bài viết rõ ràng, dễ hiểu.

Hình 4-2: Hình ảnh minh họa 2 bài post thu thập được từ forum

4.2.2 Thu thập dữ liệu từ các file đính kèm thông báo đã gửi



Hình 4-3: Hình ảnh văn bản PDF được convert từ ảnh chụp

Các thông báo gửi đến sinh viên thường đi kèm theo các tệp văn bản chi tiết như quy chế, hướng dẫn, kế hoạch học tập,... Những tệp này chủ yếu ở định dạng PDF, và đáng chú ý là phần lớn trong số đó là PDF dạng ảnh (scan từ văn bản giấy). Điều này khiến việc trích xuất văn bản trở nên khó khăn do:

- Không thể đọc text trực tiếp từ PDF dạng ảnh.
- Kết quả OCR (Optical Character Recognition) từ các công cụ thông thường có thể bị sai lệch hoặc bỏ sót nội dung.

Để đảm bảo độ chính xác trong quá trình xử lý dữ liệu, nhóm sử dụng dịch vụ chuyển đổi OCR của Google tích hợp trong Google Drive để chuyển PDF ảnh sang định dạng Google Docs/Docx. Công cụ này cho phép:

- Nhận dạng chữ từ ảnh với độ chính xác tương đối cao.

- Bảo toàn bộ cục văn bản ở mức chấp nhận được.

Tuy nhiên, do đặc thù của tài liệu scan và chất lượng ảnh khác nhau, quá trình OCR thường để lại:

- Các ký tự nhận dạng sai (ví dụ: “0” thành “O”, “I” thành “l”).
- Lỗi xuống dòng, lỗi dấu câu.
- Các phần thừa như header/footer lặp lại ở mỗi trang.

Vì vậy, admin cần rà soát và chỉnh sửa lại nội dung trước khi đưa vào pipeline xử lý và lưu trữ. Việc này giúp:

- Đảm bảo dữ liệu trong Vector Database sạch và chính xác.
- Tránh làm nhiễu kết quả truy vấn của chatbot.
- Giảm nguy cơ mô hình trả lời sai hoặc không liên quan do dữ liệu OCR lỗi.

4.2.3 Dữ liệu thu thập từ website crawl từ trang chính thức của trường

Hệ thống được tích hợp chức năng **lập lịch (scheduler)** nhằm tự động thu thập dữ liệu từ các nguồn thông báo chính thức của trường, bao gồm:

- <https://giaovu.ptit.edu.vn>
- <https://student.ptit.edu.vn/tin-tuc-parent/thong-bao/>
- <https://daotao.ptit.edu.vn/tin-tuc/tin-tuc-chung/>

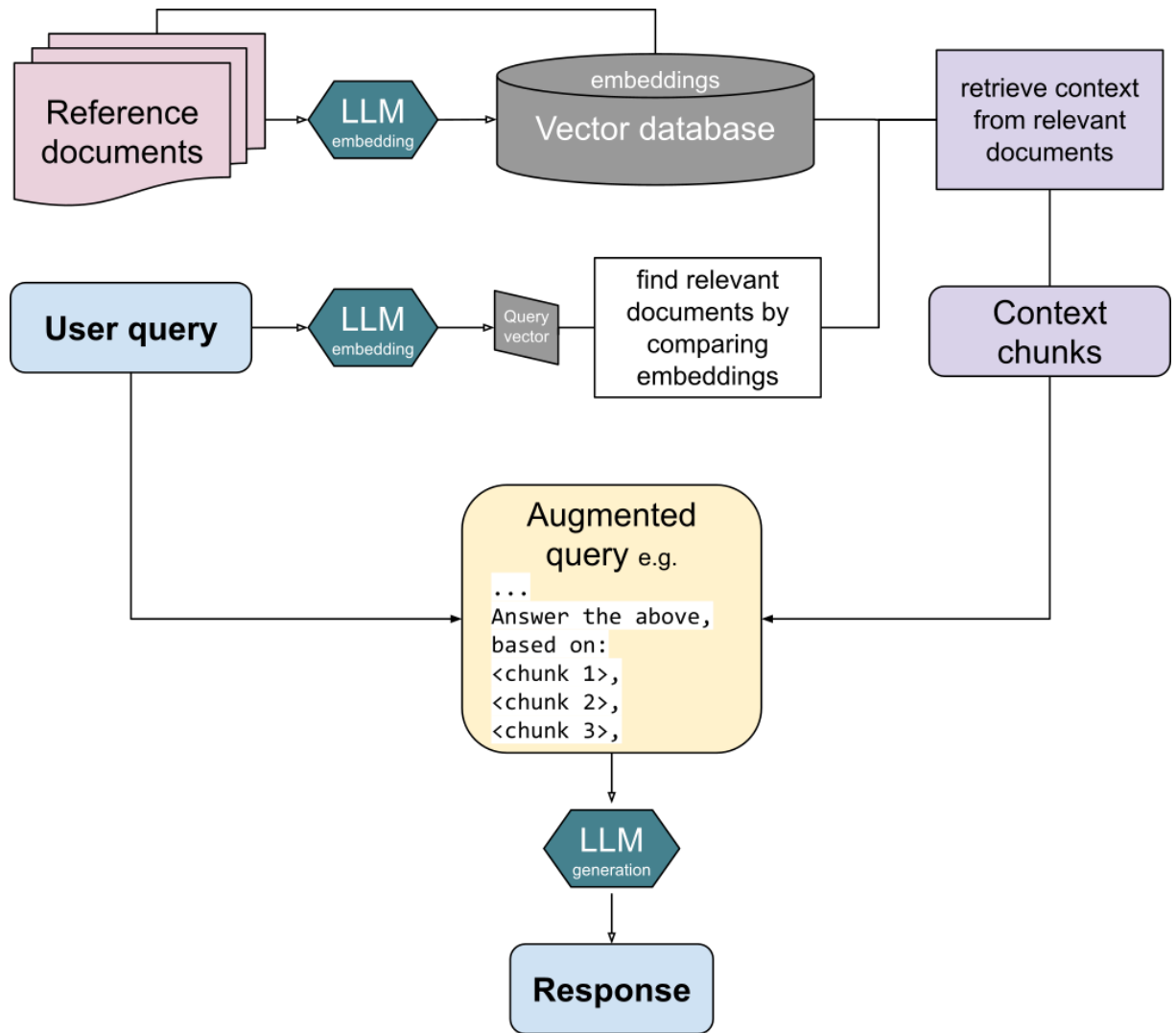
Scheduler sẽ hoạt động theo chu kỳ đã thiết lập, tự động truy cập vào các trang trên và kiểm tra xem có thông báo mới so với lần thu thập trước hay không. Khi phát hiện thông báo mới, hệ thống sẽ:

- Gửi thông báo đến admin thông qua giao diện quản trị.
- Admin có thể xem nội dung thông báo mới vừa được lấy.
- Tùy vào mức độ quan trọng và tính hữu ích, admin quyết định có đưa thông báo đó vào kho dữ liệu phục vụ AI (kho quản lý file nội dung) hay không.

Để đảm bảo hệ thống hoạt động tuân thủ và không gây ảnh hưởng đến hạ tầng của trường, tần suất quét được cấu hình phù hợp, tránh tình trạng truy cập liên tục gây quá tải (spam) website nguồn.

4.3 Module AI Chatbot

4.3.1 Luồng hoạt động tổng thể của RAG



Hình 4-4: Luồng hoạt động của hệ thống RAG

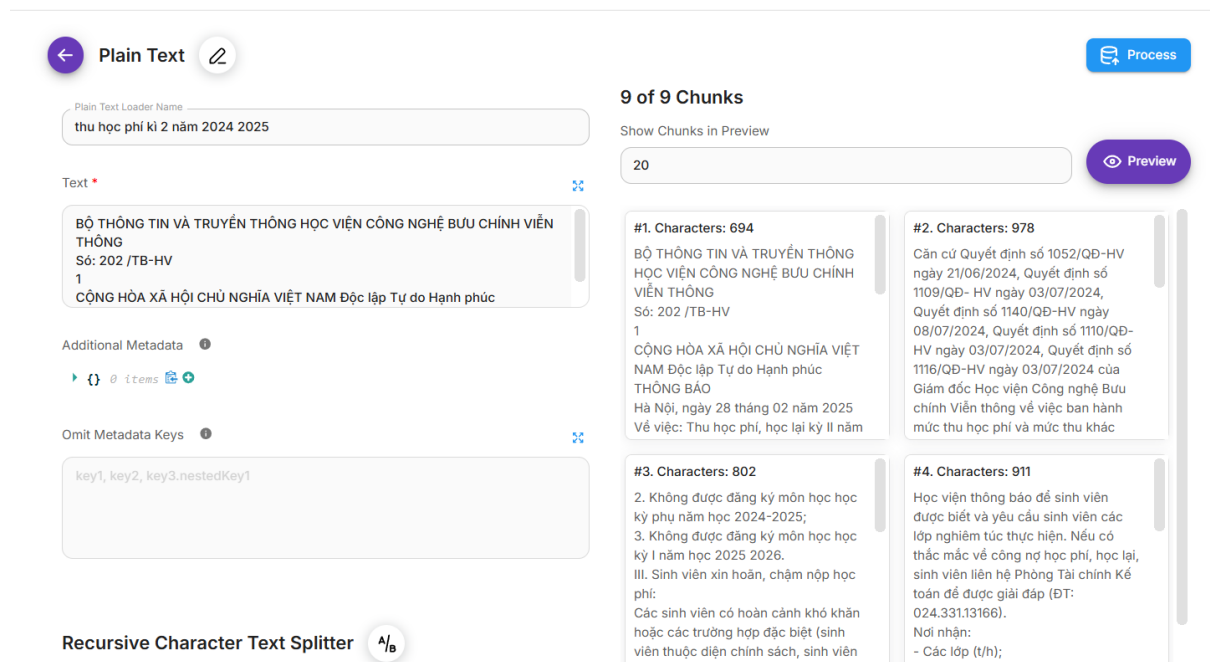
Luồng hoạt động của hệ thống RAG có thể chia thành hai giai đoạn chính:

- Pipeline chuẩn bị dữ liệu: Xử lý, chuyển đổi và lưu trữ dữ liệu vào Vector Database.
- Pipeline xử lý câu hỏi của người dùng: Truy xuất dữ liệu liên quan và sử dụng LLM để sinh câu trả lời.

4.3.2 Chi tiết luồng hoạt động RAG

4.3.2.1 Pipeline chuẩn bị dữ liệu

4.3.2.1.1 Chia nhỏ tài liệu thành các đoạn (chunking)



Hình 4-5: Hình ảnh minh họa chia tài liệu thành các chunk

Dữ liệu sau khi được thu thập về và làm sạch, cần chia nhỏ văn bản thành các đoạn (*chunks*) có độ dài phù hợp (thường từ 200 đến 1000 tokens).

Mục đích:

- Giúp thuật toán tìm kiếm dễ xác định ngữ nghĩa đoạn nội dung.
- Tăng độ chính xác khi truy xuất top-k.
- Tránh việc lãng phí dung lượng khi đưa vào LLM

Chunk size là kích thước của mỗi đoạn văn bản sau khi chia nhỏ tài liệu. Kích thước này thường được tính bằng số tokens (hoặc số ký tự, số câu tùy cách triển khai), nhưng trong các framework RAG và Flowise, token là đơn vị phổ biến hơn.

Ý nghĩa của việc lựa chọn chunk size:

Chunk quá nhỏ (ví dụ < 150 tokens) dễ làm mất bối cảnh, khiến mô hình không đủ thông tin để đưa ra câu trả lời chính xác.

Chunk quá lớn (ví dụ > 1200 tokens) khiến việc tìm kiếm trở nên kém hiệu quả, làm tăng chi phí xử lý và có thể chứa quá nhiều thông tin thừa.

Chunk size hợp lý giúp:

- Giữ trọn vẹn một ý hoặc một phần kiến thức nhất định.
- Tối ưu hóa việc index vào vector database.
- Tăng chất lượng cho bước truy xuất top-k vì các đoạn có ngữ nghĩa rõ ràng hơn.

Chunk overlap là số lượng tokens được lặp lại giữa hai chunk liên tiếp. Đây là cách giúp mô hình không bị mất thông tin khi một câu hoặc một đoạn quan trọng bị cắt giữa chừng.

Khi chunking, một câu hoặc đoạn văn dài có thể bị tách làm đôi → mất mạch ngữ nghĩa.

Chunking cho phép mô hình duy trì tính liên mạch về bối cảnh giữa các đoạn.

Tăng khả năng tìm được đúng thông tin trong tìm kiếm vector, đặc biệt với:

- văn bản quy chế,
- thông báo dài,
- văn bản PDF được OCR (dễ bị mất dấu câu).

Ví dụ:

Nếu chunk size = 500 tokens và chunk overlap = 100 tokens, thì:

- Chunk 1: tokens 0–499
- Chunk 2: tokens 400–899

-> 100 tokens cuối của chunk 1 được lặp lại ở đầu chunk 2.

Lợi ích:

- Giảm nguy cơ "trôi ngữ nghĩa".
- Đảm bảo mô hình tìm được đúng đoạn văn dù thông tin nằm ở ranh giới giữa hai chunk.
- Nâng cao độ chính xác khi trả lời.

❖ Text Splitter trong Flowise

Trong quy trình xây dựng hệ thống RAG, bước chia nhỏ văn bản (chunking) đóng vai trò then chốt trước khi dữ liệu được đưa vào Vector Database. Flowise hỗ trợ nhiều loại Text Splitter khác nhau, cho phép tùy chỉnh chunk size và chunk overlap sao cho phù hợp với đặc thù từng loại tài liệu. Việc lựa chọn đúng splitter giúp giữ được bối cảnh nội dung, giảm lỗi cắt câu giữa chừng và nâng cao độ chính xác truy xuất thông tin.

Flowise cung cấp các Text Splitter dựa trên thư viện của **LangChain**

Một số Text Splitter phổ biến như:

- **Recursive Character Text Splitter**: là splitter mặc định và được sử dụng nhiều nhất. Cơ chế hoạt động là thử chia văn bản theo nhiều cấp độ khác nhau như đoạn, câu và từ. Nhờ đó, hệ thống hạn chế được tình trạng cắt câu không tự nhiên, thông báo từ trang web trường, hoặc văn bản hành chính.
- **Character Text Splitter**: chia văn bản dựa trên số lượng ký tự cố định. Splitter này thích hợp cho các dữ liệu được chuyển từ OCR, nơi dấu câu có

thể bị mất hoặc nhận dạng sai. Việc chia theo ký tự giúp đảm bảo tính ổn định khi văn bản không còn cấu trúc rõ ràng.

- **Markdown Splitter:** dành cho các tài liệu được viết dưới dạng Markdown. Splitter này tận dụng cấu trúc tiêu đề, bullet points và thứ bậc nội dung để tạo ra các chunk có ý nghĩa.
- **HtmlToMarkdown Text Splitter:** dùng để xử lý dữ liệu thu thập từ các trang web có định dạng HTML. Splitter sẽ loại bỏ thẻ HTML không cần thiết và chỉ giữ lại phần nội dung chính, giúp cho việc embedding chính xác hơn.
- **Token Text Splitter:** chia văn bản dựa trên số lượng tokens, phù hợp khi cần đồng bộ với cách LLM xử lý dữ liệu. Điều này giúp kiểm soát chính xác kích thước ngữ cảnh, đặc biệt quan trọng khi làm việc với các mô hình giới hạn số token đầu vào.

❖ Chia chunk khi sử dụng data thực tế từ forum, file đính kèm thông báo, dữ liệu crawl từ website trường

a) Dữ liệu từ forum

Dữ liệu thu thập từ forum sẽ được chuyển đổi sang định dạng Markdown để thuận tiện cho việc xử lý. Các bài viết sau khi được admin chọn lọc — bao gồm tiêu đề, nội dung câu hỏi và các bình luận hữu ích — sẽ được tổng hợp vào một file duy nhất. Trong file Markdown, mỗi bài đăng được đánh dấu bằng ký tự “#” để tạo thành các phân đoạn (section) rõ ràng. Khi đưa vào Flowise, việc sử dụng **Markdown Text Splitter** cho phép hệ thống tự động chia dữ liệu dựa theo các header này.

Markdown Splitter có khả năng tách chunk linh hoạt theo các cấp độ header “#”. Điều này có nghĩa là dù nội dung một bài đăng ngắn và chưa đạt đến *chunk size*, splitter vẫn tạo thành một chunk riêng biệt, giúp phân tách hoàn toàn nội dung của từng bài viết. Nhờ đó, dữ liệu không bị trộn lẫn và từng bài đăng được giữ nguyên cấu trúc.

Khi test thực tế việc thiết lập chunk size từ 600–800 ký tự kết hợp với chunk overlap = 200 giúp hạn chế việc mất ngữ cảnh, đặc biệt tại các đoạn chuyển tiếp giữa tiêu đề, phần hỏi và phần bình luận. Do đa số bài viết trên forum có độ dài vừa phải, ngưỡng 800 ký tự giúp **Markdown Splitter không gộp nhầm hai bài đăng khác nhau** vào cùng một chunk và cũng tránh tạo ra các đoạn quá dài, vốn dễ gây nhiễu hoặc khiến mô hình LLM khó nắm bắt ngữ nghĩa. Nhờ đó, mô hình có thể hiểu chính xác hơn nội dung và mối liên hệ giữa các phần trong bài viết.

Cách chia chunk này đảm bảo mô hình RAG có thể truy xuất thông tin chính xác, đầy đủ ngữ cảnh và tối ưu hơn trong quá trình trả lời câu hỏi của sinh viên.

Format lưu bài đăng:

#Tiêu đề bài viết 1, nội dung bài viết

Comment 1

Comment 2

#Tiêu đề bài viết 2, nội dung bài viết

Comment 1

Comment 2

b) Dữ liệu từ file pdf

Đối với các file PDF, đặc biệt là những PDF dạng ảnh, sau khi chuyển đổi bằng OCR, nội dung thường được tách thành nhiều đoạn nhỏ và ngắt dòng liên tục bằng ký tự `\n` hoặc theo dấu “.”. Do đặc thù văn bản sau OCR không còn giữ được cấu trúc tiêu đề hay mục lục rõ ràng, nhóm lựa chọn sử dụng **Recursive Character Text Splitter** để chia chunk.

Cách tách này giúp phân chia văn bản dựa trên mức độ ưu tiên: tách theo dấu câu → xuống dòng → độ dài ký tự. Nhờ đó, dữ liệu sau OCR được chia thành các chunk mạch lạc hơn, hạn chế việc cắt ngang câu và bảo toàn ngữ nghĩa tốt hơn so với các phương pháp splitter khác.

c) Dữ liệu crawl từ website trường

Dữ liệu được thu thập từ website của trường thường chứa nhiều thẻ HTML như `<p>`, `<a>`, `<div>`,... vốn không cần thiết cho quá trình xây dựng tri thức. Các thẻ này nếu giữ nguyên sẽ gây nhiễu, làm giảm chất lượng khi trích xuất nội dung và tạo embedding.

Để xử lý vấn đề này, nhóm sử dụng **HtmlToMarkdown Text Splitter** trong Flowise nhằm chuyển đổi HTML sang định dạng Markdown. Qua bước này:

- Các thẻ HTML được loại bỏ hoặc chuyển đổi sang cấu trúc Markdown tương ứng.
- Nội dung văn bản được giữ nguyên, sạch và dễ đọc hơn.
- Markdown giúp bảo toàn những cấu trúc có ý nghĩa như tiêu đề (`#`, `##`), danh sách (`-`, `*`), liên kết, đoạn văn,...

Sau khi chuyển sang Markdown, dữ liệu được tiếp tục chia nhỏ tương tự **Markdown Text Splitter**. Bộ chia này tận dụng chính cấu trúc phân cấp của Markdown để tách tài liệu thành những **chunk có ngữ nghĩa rõ ràng**, chẳng hạn

như từng mục nhỏ, từng phần thông báo hoặc từng nội dung liên quan. Điều này giúp:

- Hạn chế mất ngữ cảnh khi embedding.
- Các chunk đại diện tốt hơn cho một ý hoàn chỉnh.
- Cải thiện kết quả truy vấn trong giai đoạn Retrieval của RAG.

4.3.2.1.2 Tạo embedding cho từng chunk

Embedding là biểu diễn dạng vector của văn bản. Hệ thống sử dụng mô hình embedding (như OpenAI text-embedding, BERT, Instructor...) để chuyển mỗi chunk thành một vector nhiều chiều.

Embedding có vai trò:

- Nắm bắt ngữ nghĩa đoạn văn.
- Giúp so sánh mức độ tương tự thông qua cosine similarity
- Tạo điều kiện cho truy xuất văn bản với độ chính xác cao.

Nhóm lựa chọn mô hình embedding gemini-embedding-exp-03-07 cho hệ thống chatbot vì các ưu điểm nổi bật sau:

- Khả năng nắm bắt ngữ nghĩa vượt trội, gemini-embedding-exp-03-07 được tối ưu hóa cho các tác vụ hiểu ngữ cảnh và quan hệ giữa các câu trong văn bản.
- Tối ưu cho đa ngôn ngữ, đặc biệt là tiếng Việt, gemini-embedding-exp-03-07 được huấn luyện trên tập dữ liệu đa ngôn ngữ quy mô lớn, giúp mô hình hiểu tốt hơn các đặc trưng ngôn ngữ và cú pháp tiếng Việt. Nhờ đó, chatbot có khả năng phân tích ý định người dùng, nhận diện ngữ cảnh và truy xuất thông tin chính xác hơn so với các mô hình embedding chủ yếu tối ưu cho tiếng Anh.

4.3.2.1.3 Lưu trữ embedding vào Vector Database

Các vector embedding sau khi được tạo ra sẽ được lưu trữ trong Vector Database cùng với phần metadata tương ứng. Metadata có thể bao gồm: nội dung gốc của chunk, nguồn tài liệu (forum, PDF, website), tiêu đề bài viết, ... giúp quá trình truy xuất trở nên chính xác hơn.

Pinecone hỗ trợ lưu vector embedding theo index. Mỗi index tương ứng với một database lưu nhiều bản ghi dữ liệu. Mỗi bản ghi dữ liệu sẽ bao gồm id, vector, metadata

Record



namespace: demo04

ID: 01fa7430-5fc2-4c47-905b-9daaedf4920c

values: [0.00487735076, -0.0197011381, -0.000382859085, -0.0450623669, 0.0002020...

metadata:

docId: "27b936a0-9fbe-43a8-8dc2-9297836861a1"

loc.lines.from: 366

loc.lines.to: 380

text: "Viện CDIT\n0936133144\n83\nD24CQGA02-B Cao Minh Thắng\nViện
CDIT\n0936133144\n84\nD24CQGA03-B Nguyễn Đức Hoàng\nViện
CDIT\n0904371818\n85\nD24CQGA04-B Nguyễn Đức Hoàng\nViện
CDIT\n0904371818\nDanh sách gồm 85 lớp"

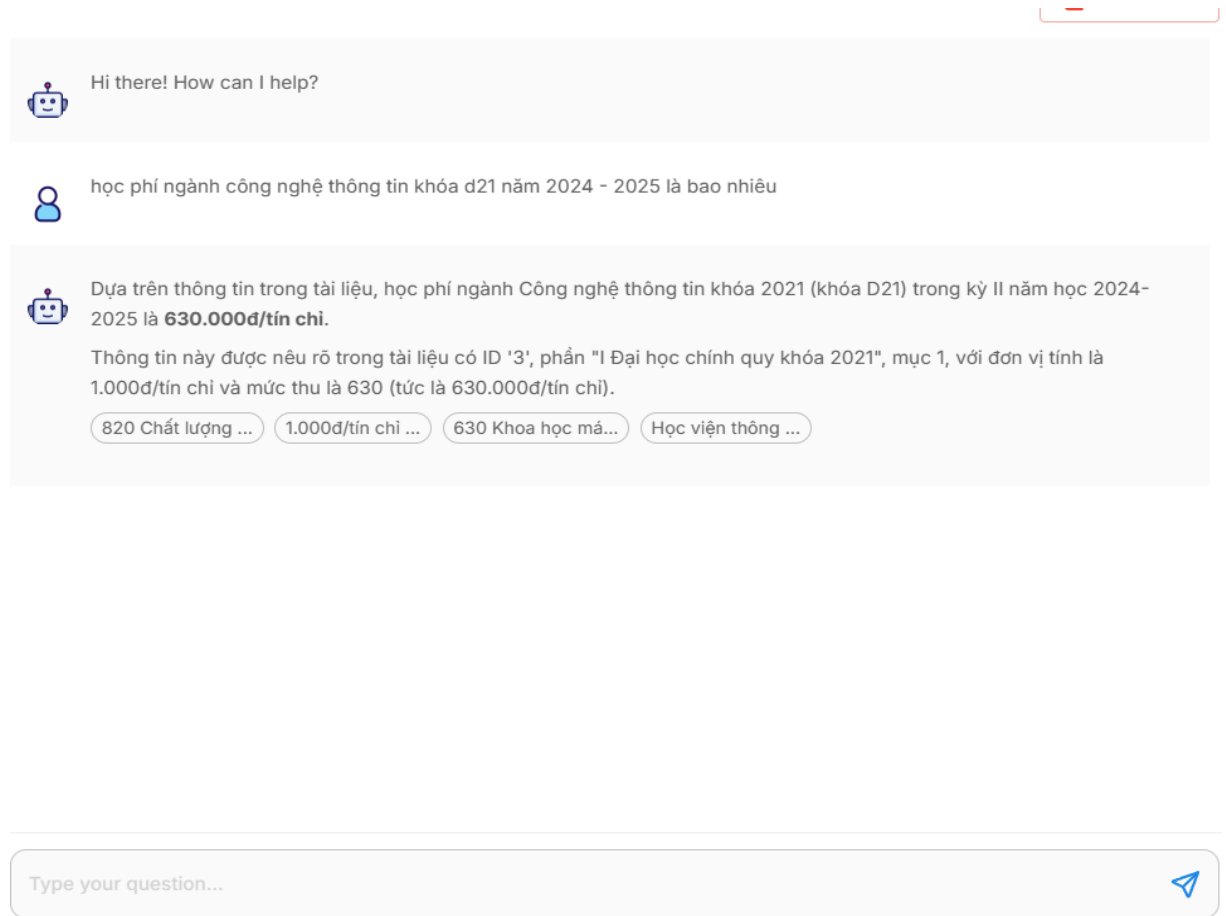
[Edit record](#)

[Close](#)

Hình 4-6: Bản ghi dữ liệu mẫu

Database để lưu trữ vector được cấu hình là lưu vector 3072 chiều để phù hợp với kết quả trả về của model gemini-embedding-exp-03-07

4.3.2.2 Pipeline xử lý câu hỏi của người dùng



Hình 4-7: Ảnh minh họa câu hỏi với chatbot AI

Source Documents

```

{ 3 items
  pageContent :
    "820 Chất lượng cao khóa 2023 trở về trước 1 Đại học chính quy chất lượng cao ngành Công nghệ thông tin khóa 2023 trở về trước 1.000đ/tín chỉ 1.200 Đại học chính quy chất lượng cao ngành marketing 2 1.000đ/tín chỉ 1.100 khóa 2023 Chất lượng cao khóa 2024 1 Chất lượng cao ngành Công nghệ thông tin 1.000đ/tín chỉ 1.250 2 Đại học chính quy chất lượng cao ngành marketing 1.000đ/tín chỉ 1.150 3 Ngành kế toán chuẩn quốc tế ACCA 1.000đ/tín chỉ 1.150 4 Chương trình thiết kế và Phát triển game 1.000đ/tín chỉ 1.000 5 Chương trình Công nghệ thông tin Việt-Nhật 6 II. - Mức thu học phí sinh viên Lào diện tự túc Lưu ý: Đối với sinh viên khóa 2021,2022,2023,2024 học phí phải nộp kỳ II năm học 2024-2025 căn cứ vào tổng số tín chỉ sinh viên đăng ký theo kế hoạch và chương trình đào tạo được Học viện phê duyệt. Mức thu học lại áp dụng cho hệ Đại học chính quy, văn bằng 2 chính quy 1.000đ/tín chỉ 980 1.000đ/ tháng 3.350 VIỆ TT Hệ đào tạo Đơn vị tính Mức thu NG CHÍNH I"
  metadata : { 3 items
    docId : "8a181026-983a-41a6-af91-9d42d9143433"
    loc.lines.from : 110
    loc.lines.to : 154
  }
  id : "4d1b2f44-048d-436c-8e1a-6f550f64eb95"
}

```

Hình 4-8: Ảnh minh họa source lấy từ vector database để làm thông tin cho AI trả lời

Khi người dùng nhập một câu hỏi, ví dụ: “học phí ngành công nghệ thông tin khóa d21 năm 2024 - 2025 là bao nhiêu”, hệ thống bắt đầu thực hiện giai đoạn truy xuất thông tin trong kiến trúc RAG.

Đầu tiên, câu hỏi của người dùng được chuyển đổi thành một vector embedding bằng cùng mô hình embedding đã được sử dụng trong quá trình nạp dữ liệu vào Vector Database. Việc sử dụng chung một mô hình giúp đảm bảo tính đồng nhất trong không gian vector, từ đó tăng độ chính xác khi tính toán mức độ tương đồng (similarity) giữa câu hỏi và các đoạn dữ liệu đã lưu trữ.

Sau khi có embedding của câu hỏi, hệ thống tiến hành truy vấn Vector Database (trong trường hợp này là Pinecone) để tìm ra bốn đoạn văn bản (top-4) có ngữ nghĩa gần nhất dựa trên độ tương đồng cosine. Đây là những đoạn dữ liệu được xem là phù hợp nhất để cung cấp thông tin cho mô hình sinh ngôn ngữ.

Các đoạn văn bản được truy xuất sẽ được kết hợp với một prompt được thiết kế sẵn để cung cấp ngữ cảnh đầy đủ cho mô hình LLM. Prompt bao gồm:

- Nội dung các đoạn văn bản được truy xuất
- Câu hỏi của người dùng
- Các chỉ dẫn yêu cầu mô hình trả lời dựa trên tài liệu và hạn chế suy luận ngoài ngữ cảnh

Mô hình LLM sau đó sử dụng phần ngữ cảnh này để phân tích, tổng hợp và diễn đạt lại thông tin một cách rõ ràng, mạch lạc, đúng với tài liệu gốc. Kết quả cuối cùng là câu trả lời được tối ưu hóa về mặt ngôn ngữ nhưng vẫn đảm bảo tính chính xác về nội dung.

4.3.3 Đánh giá độ chính xác và các mô hình LLM đã thử nghiệm

4.3.3.1 Đánh giá độ chính xác

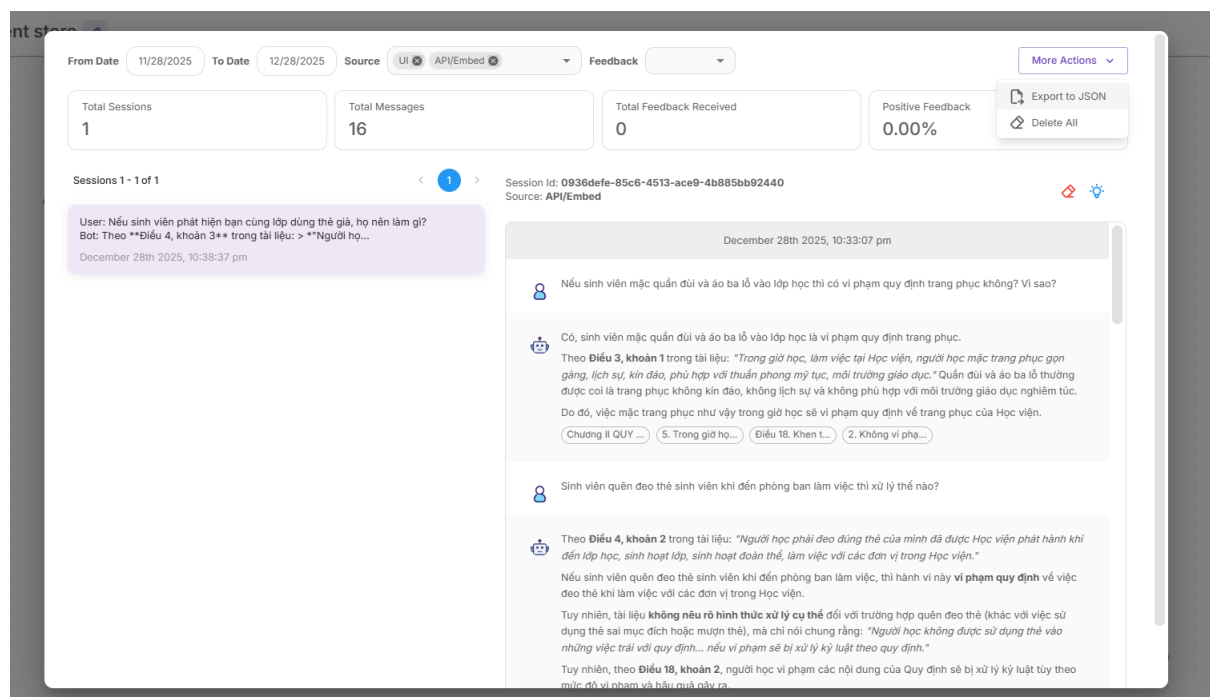
Để kiểm tra mức độ chính xác của chatbot RAG, nhóm sử dụng mô hình ngôn ngữ lớn **Gemini 3 Pro** làm “giám khảo” để đánh giá chất lượng câu trả lời mà chatbot RAG sinh ra.

Gemini 3 pro được sử dụng làm mô hình giám khảo vì sở hữu khả năng đánh giá ngữ nghĩa sâu, độ nhất quán và tính chính xác của câu trả lời tốt hơn các thế hệ trước. Mô hình này được tối ưu cho các tác vụ đánh giá chất lượng phản hồi (LLM-as-a-judge) với độ ổn định cao và ít thiên vị hơn. Nhờ dung lượng lớn và khả năng reasoning mạnh, Gemini 3 pro có thể phân tích bối cảnh RAG phức tạp và so sánh câu trả lời một cách khách quan. Điều này giúp đảm bảo việc đánh giá kết quả từ các mô hình Gemini thấp hơn và Qwen 32B được tin cậy và nhất quán.

Quy trình đánh giá gồm ba thành phần đầu vào:

- Câu hỏi của người dùng
- Câu trả lời được sinh bởi chatbot RAG
- Đoạn dữ liệu truy xuất từ Vector Store (top-k kết quả được RAG lấy ra)

Các dữ liệu đầu vào lấy từ lịch sử cuộc trò chuyện của người dùng với chatbot RAG



Hình 4-9: Hình ảnh minh họa lịch sử cuộc trò chuyện

Văn bản chính được sử dụng làm nội dung chính cho bài test là 191.QD-Van-hoa-hoc-duong-2023.pdf

<https://drive.google.com/file/d/1FXT77fCiztxeXKEqmaCxUR3k0Z5YgySL/view?usp=sharing>

+	STT	Câu hỏi (User)	Câu trả lời (Chatbot)	Nguồn tài liệu (Vector Database)	Đánh giá của Gemini
	1	Người học phải mặc trang phục như thế nào khi đến lớp	Mặc gọn gàng, lịch sự, kín đáo. Khuyến khích mặc áo có logo trường/đồng phục. Môn GDQP, GDTC có quy định riêng.	Điều 3: "người học mặc trang phục gọn gàng... Khuyến khích... mặc áo có logo... Trong giờ học các môn Giáo dục quốc phòng... sử dụng trang phục theo quy định riêng."	Chính xác. Bot trích xuất đúng và đủ ý từ Điều 3.
	2	Người học có bắt buộc phải mặc áo đồng phục của Học viện không?	Không bắt buộc, nhưng được khuyến khích trong các hoạt động tập thể và dịp học.	Điều 3, khoản 2: "Khuyến khích người học mặc áo có logo... hoặc áo đồng phục..."	Chính xác. Bot hiểu đúng từ "Khuyến khích" nghĩa là không bắt buộc.
	3	Khi bị mất hoặc hỏng thẻ sinh viên thì phải làm gì?	Báo ngay cho Phòng Giáo vụ để làm thủ tục cấp lại.	Điều 4, khoản 3: "Trong trường hợp mất thẻ, hỏng thẻ, phải báo ngay cho Phòng Giáo vụ..."	Chính xác. Thông tin hoàn toàn trùng khớp.
	4	Người học có	Không. Phải bảo quản	Điều 4, khoản 3: "...không	Chính xác.

Hình 4-10: Hình ảnh minh họa kết quả đánh giá

Kết quả chi tiết:

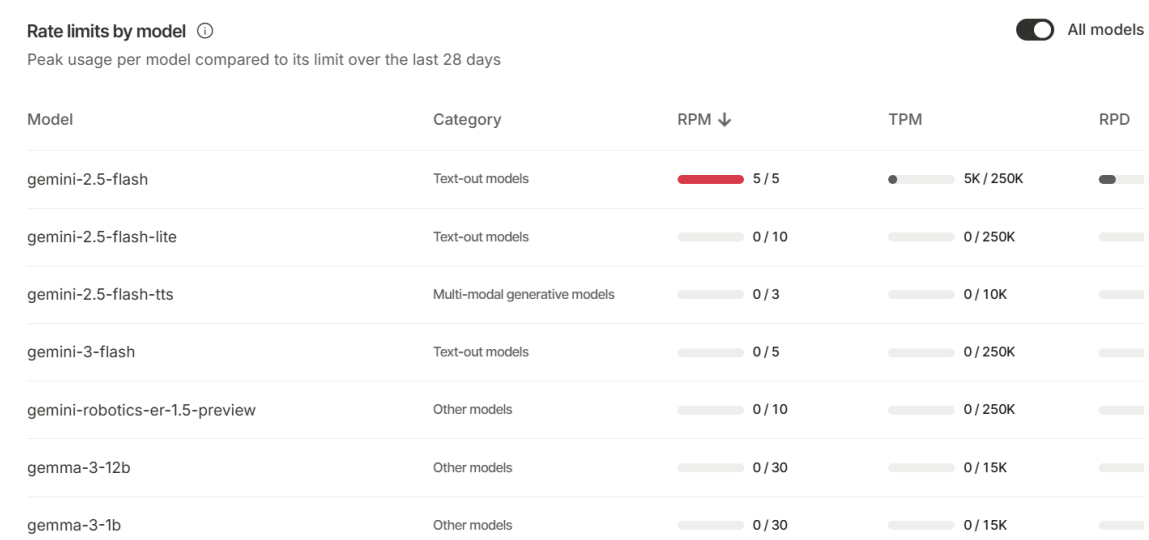
<https://docs.google.com/document/d/1iNFenWA7uNFJJ0oqSLB9TBrGB8bkP2lzIGsEUT3t4sE/edit?usp=sharing>.

- **Nhận xét chung:** câu trả lời của LLM tương đối chính xác với câu hỏi và nội dung trong văn bản

4.3.3.2 Chi phí

Hiện tại nhóm đã thử nghiệm các mô hình LLM như gemini: gemini-2.5-flash, gemini-2.5-flash-lite, Qwen/Qwen3-VL-32B-I triển khai trên huggingface

1. Gemini



Hình 4-11: Hình ảnh minh họa các model của gemini

Mô hình Gemini thể hiện hiệu quả cao khi áp dụng vào bài toán RAG nhờ khả năng hiểu ngữ nghĩa mạnh, độ chính xác cao khi truy vấn thông tin và khả năng tổng hợp câu trả lời tự nhiên. Gemini hỗ trợ tốt việc mã hoá văn bản (embeddings) với chất lượng vector ổn định, giúp tăng độ phù hợp khi truy xuất dữ liệu từ Vector Database.

Tuy nhiên mô hình Gemini chi phí khá cao, và số lượng token dùng thử ít.

2. Qwen/Qwen3-VL-32B-I

Qwen3-VL-32B-Instruct là mô hình đa phương thức dung lượng lớn thuộc thể hệ Qwen3 của Alibaba, được tối ưu cho cả văn bản và hình ảnh. Mô hình có khả năng hiểu ngữ cảnh sâu, phân tích chi tiết và suy luận logic mạnh, phù hợp cho các tác vụ hỏi–đáp phức tạp. Nhờ kiến trúc tối ưu và khả năng xử lý đầu vào dài, Qwen3-VL-32B-I cho chất lượng phản hồi ổn định trong các hệ thống RAG. Đây là một trong những mô hình open-source mạnh nhất hiện nay, vừa hiệu năng cao vừa linh hoạt khi triển khai. Mô hình cũng hỗ trợ tinh chỉnh và tùy biến, đáp ứng tốt nhu cầu nghiên cứu và phát triển trong môi trường học thuật.

3. So sánh 3 mô hình đã thử

Tiêu chí	Gemini-2.5-Flash	Gemini-2.5-Flash-Lite	Qwen3-VL-32B-Instruct
Khả năng đọc hiểu & suy luận tiếng Việt	Tốt, ổn định, hiểu ngữ cảnh mạnh	Khá tốt nhưng thấp hơn Flash, phù hợp câu ngắn, trong thực tế nếu văn bản chunk trả về quá nhiều thông tin thì sẽ không suy luận được	Rất tốt, tối ưu cho tiếng Việt, suy luận chi tiết
Chất lượng trả lời trong RAG	Cao, giữ ngữ cảnh tốt	Trung bình–khá, phù hợp ứng dụng nhẹ	Cao, đặc biệt với câu trả lời dài hoặc phức tạp
Tốc độ phản hồi	Nhanh	Rất nhanh (nhanh nhất)	Chậm hơn hai mô hình Gemini do kích thước lớn
Chi phí sử dụng	Trung bình	Rẻ nhất trong 3 mô hình	Rẻ hơn đáng kể khi dùng qua Hugging Face hoặc self-host
Phù hợp sử dụng	RAG yêu cầu chất lượng cao	RAG nhẹ, chi phí thấp, tốc độ ưu tiên	RAG cần trả lời chi tiết, hỗ trợ tiếng Việt mạnh, ngân sách thấp

Bảng 4-1: Bảng so sánh các mô hình đã chọn

CHƯƠNG V: Cài đặt và Triển khai (Implementation)

5.1 Cài đặt dự án :

5.1.1 Frontend:

5.1.1.1 Yêu cầu môi trường (Prerequisites):

- Node.js: Phiên bản 18.17.0 trở lên (Khuyến nghị bản LTS mới nhất để tương thích tốt với Next.js App Router).
- Package Manager: NPM (đi kèm Node.js) hoặc Yarn/PNPM.
- IDE: Visual Studio Code (Khuyến nghị) hoặc WebStorm.

Các bước cài đặt và triển khai:

Bước 1: Clone mã nguồn và cài đặt thư viện:

- Mở Terminal hoặc Command Prompt, di chuyển đến thư mục muốn lưu trữ và chạy lệnh:
 - git clone <https://github.com/Anhuan162/graduation-project.git>
 - cd graduation-project/frontend // (Lưu ý: trở đúng vào folder chứa code FE)
- Cài đặt các gói phụ thuộc (Dependencies) được khai báo trong **package.json**, chạy lệnh
 - npm install
- Cấu hình biến môi trường
 - Tạo file .env.local tại thư mục gốc của dự án Frontend
 - Thiết lập các tham số kết nối tới Backend và AI Service (Flowise)

Bước 2: Kết nối tới Backend Spring Boot

NEXT_PUBLIC_API_URL=http://localhost:8080/api/v1

Bước 3: Cấu hình Chatbot AI (Flowise)

NEXT_PUBLIC_FLOWISE_API_URL=https://cloud.flowiseai.com/api/v1/prediction/

NEXT_PUBLIC_CHATFLOW_ID=9d4b2c4a-572c-48df-a92e-c3c94a80dbff

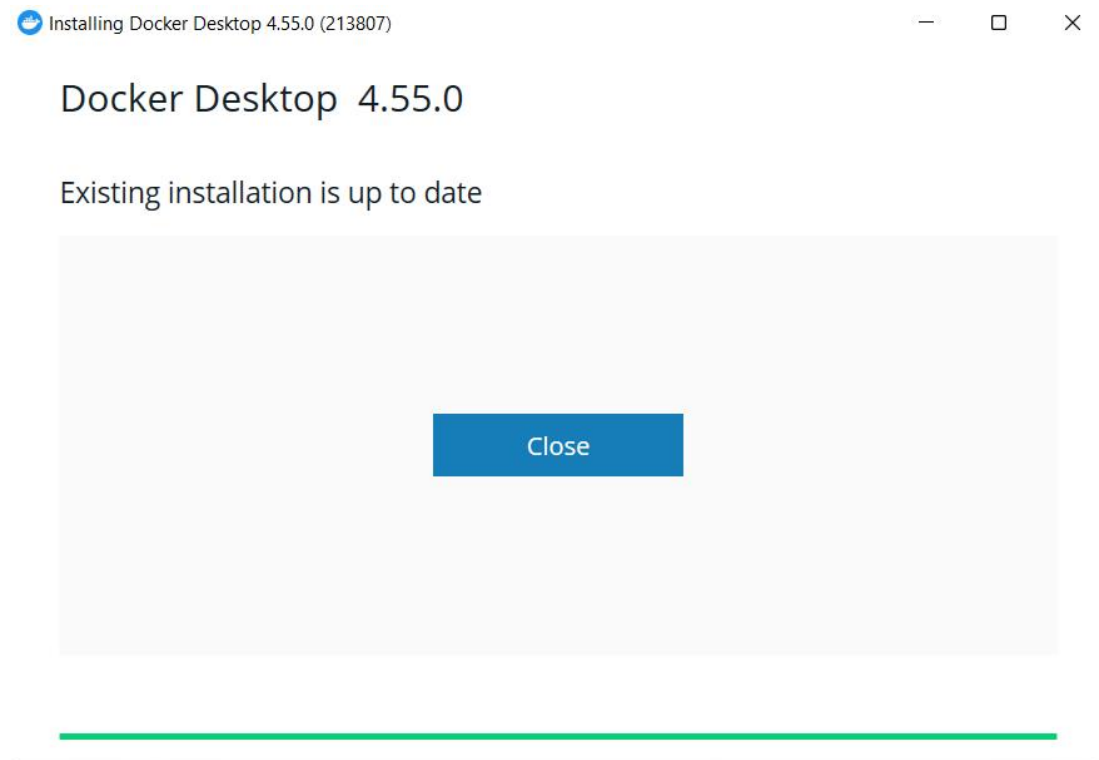
Bước 4: Khởi chạy ứng dụng

- Chạy ứng dụng ở chế độ phát triển (Development Mode):
 - npm run dev
- Truy cập trình duyệt tại địa chỉ: http://localhost:3000

5.1.2 Backend:

5.1.2.1 Cài đặt Docker :

Truy cập trang web <https://www.docker.com/products/docker-desktop/> rồi tải xuống docker desktop . Sau đó cài đặt theo mặc định :



Hình 5-1: Minh họa cài đặt docker desktop

5.1.2.2 Cài đặt postgresQL trên Docker:

Bật command prompt và chạy lệnh :

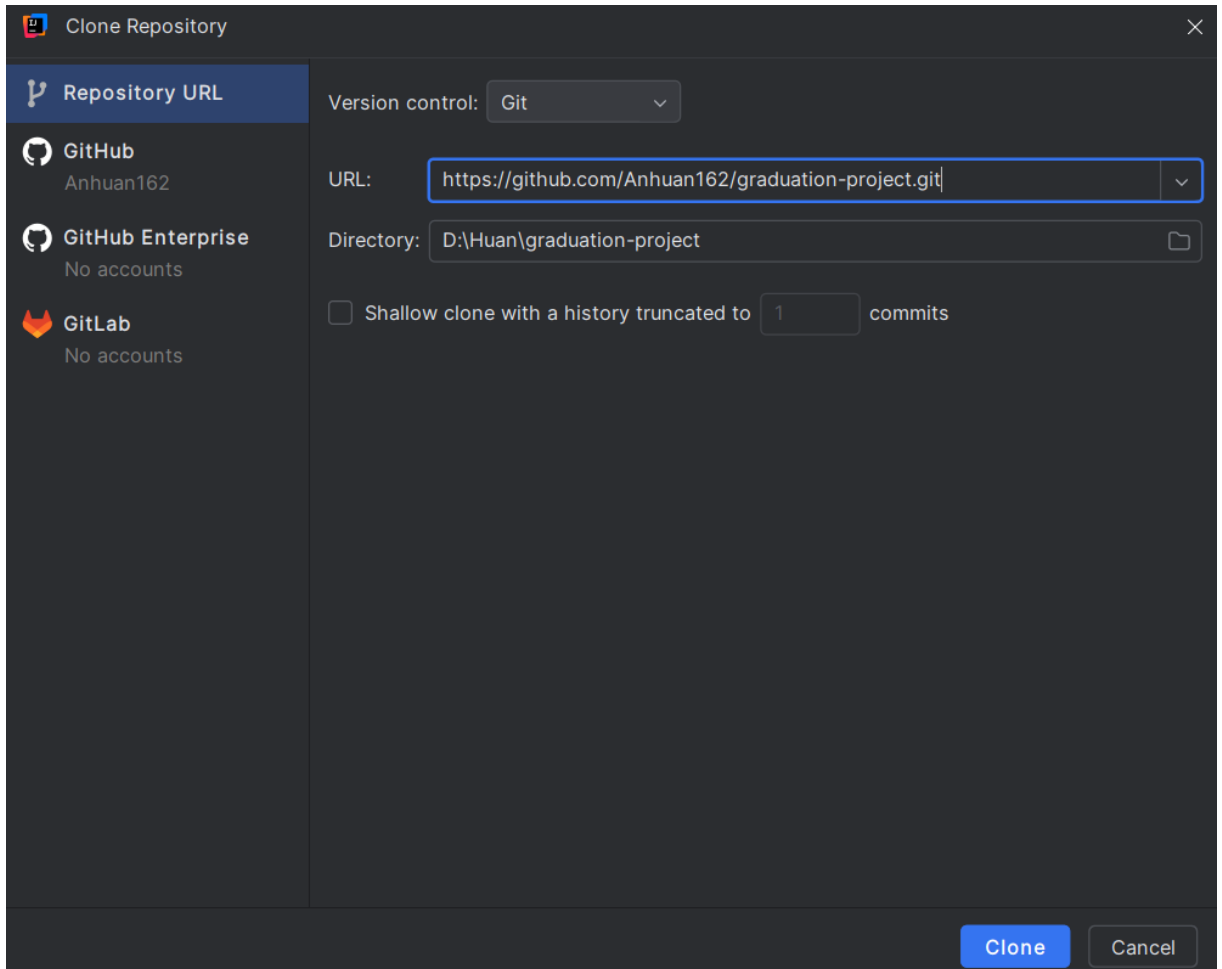
```
docker run -d --name postgres-14 -e POSTGRES_USER=postgres -e  
POSTGRES_PASSWORD=root -e POSTGRES_DB=authdb -p  
5432:5432 postgres:14
```

5.1.2.3 Cài đặt Redis Stream trên Docker:

- Bật cmd và chạy lệnh :
`docker run --name redis-lab -p 6379:6379 -d redis`
- Cài đặt DBeaver và IntelliJ Idea:
- Download và cài đặt DBeaver từ đường dẫn : <https://dbeaver.io/download/>
- Download và cài đặt IntelliJ Idea Ultimate từ đường dẫn :
<https://www.jetbrains.com/idea/download/?section=windows>
- Clone dự án và chạy:

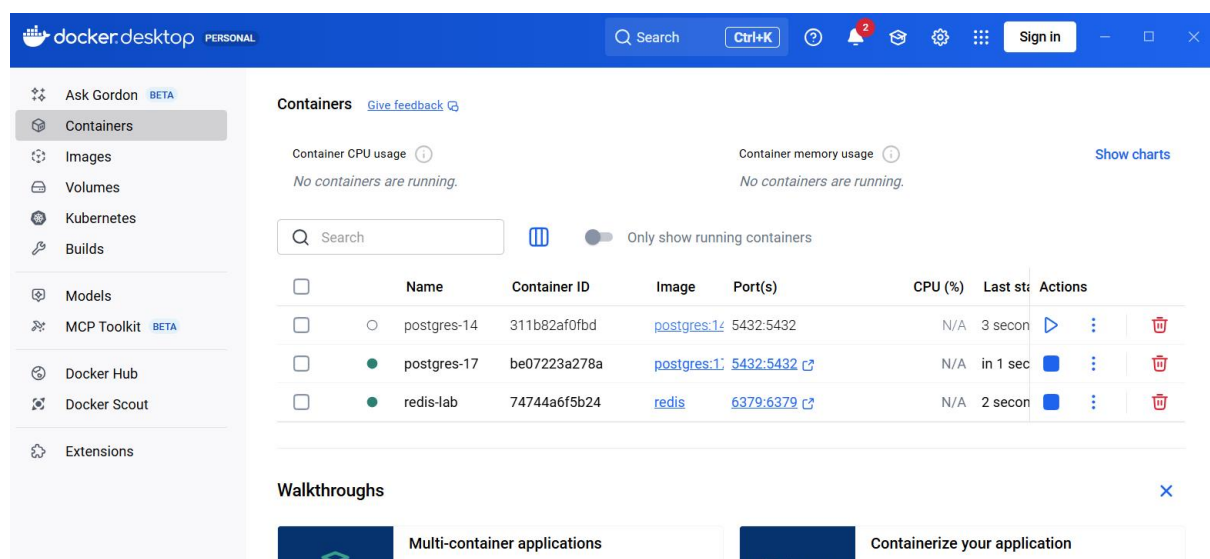
Chọn **File** ở góc trái giao diện IntelliJ Idea, chọn **New** và cuối cùng là **Project from Version Control**. Sau đó, gắn đường dẫn sau vào ô trống rồi chọn clone để kéo project về:

<https://github.com/Anhuan162/graduation-project.git>



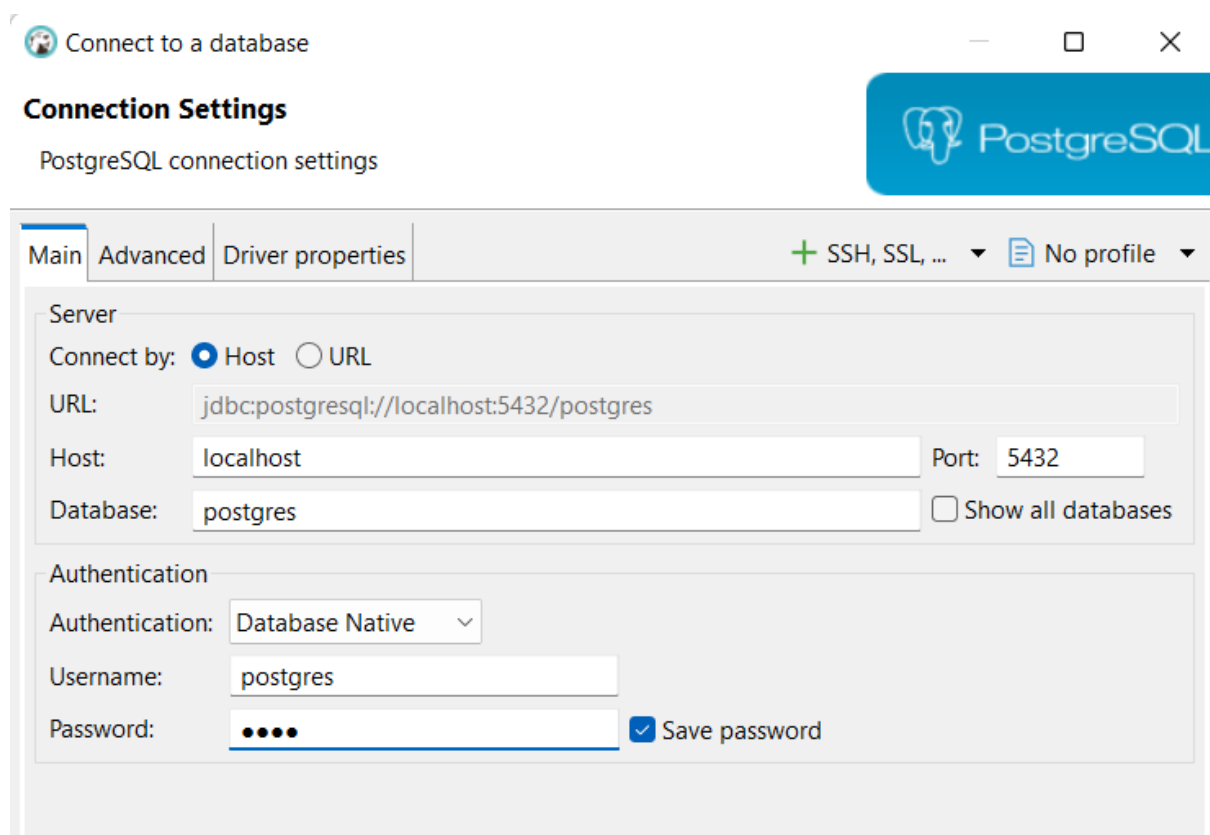
Hình 5-2: Minh họa clone project

Chạy lên 2 container như hình trong Docker Desktop:



Hình 5-3: Minh họa chạy containers

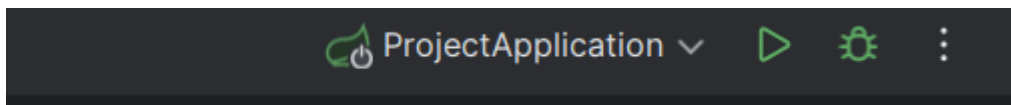
Bật DBeaver và tạo connection với username và password nằm ở câu lệnh cài đặt postgresSQL ở trên :



Hình 5-4: Minh họa cài đặt database

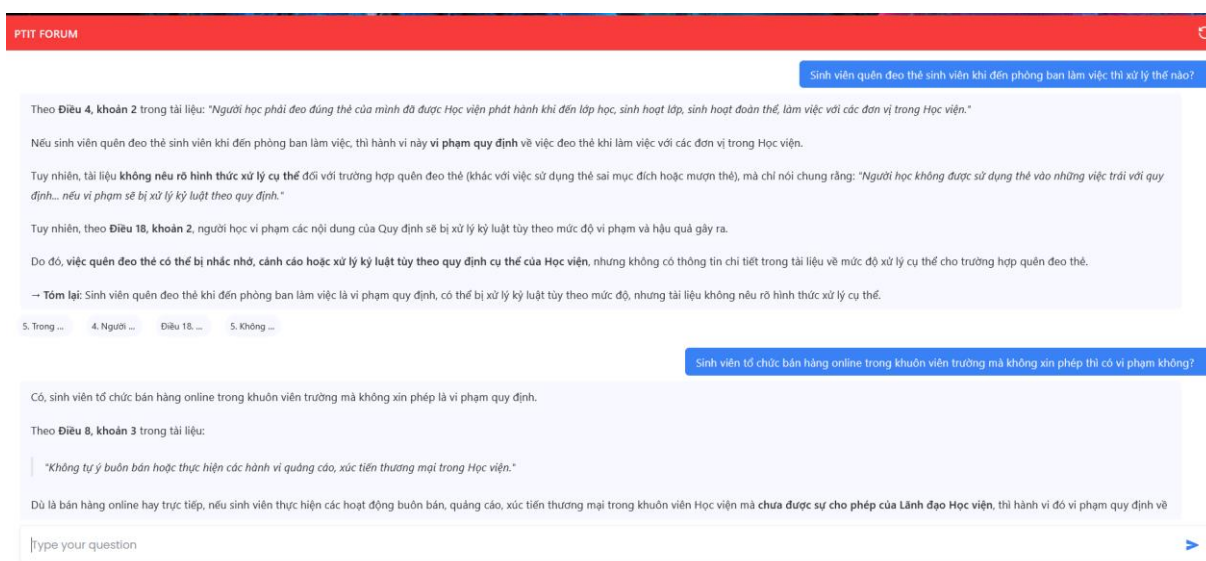
Tiếp theo tạo Database mới có tên : authdb

Cuối cùng run project trong IntelliJ Idea

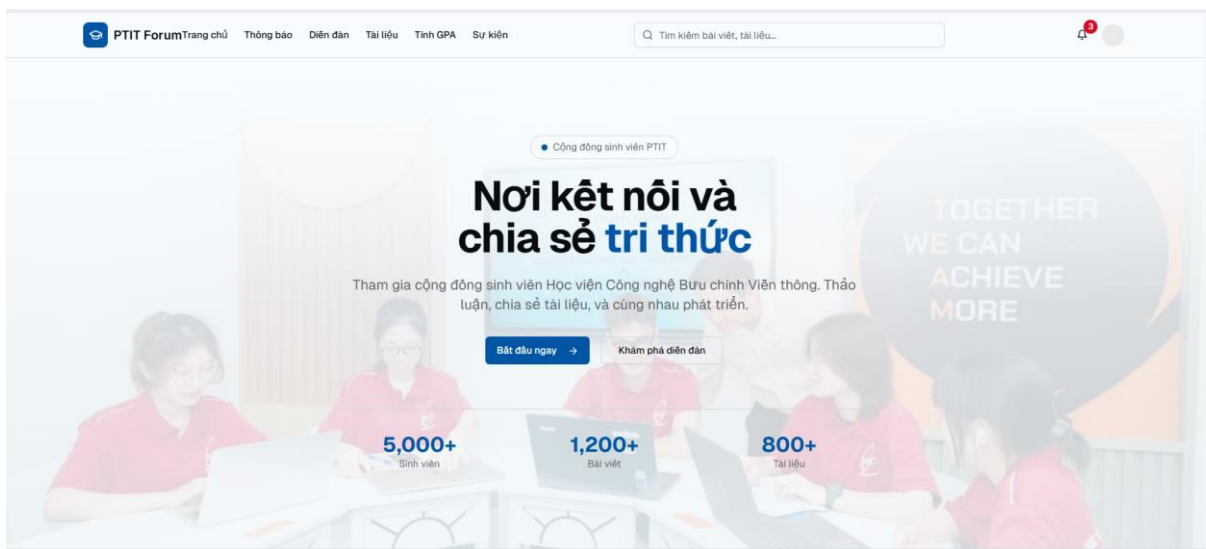


Hình 5-5: Chạy dự án

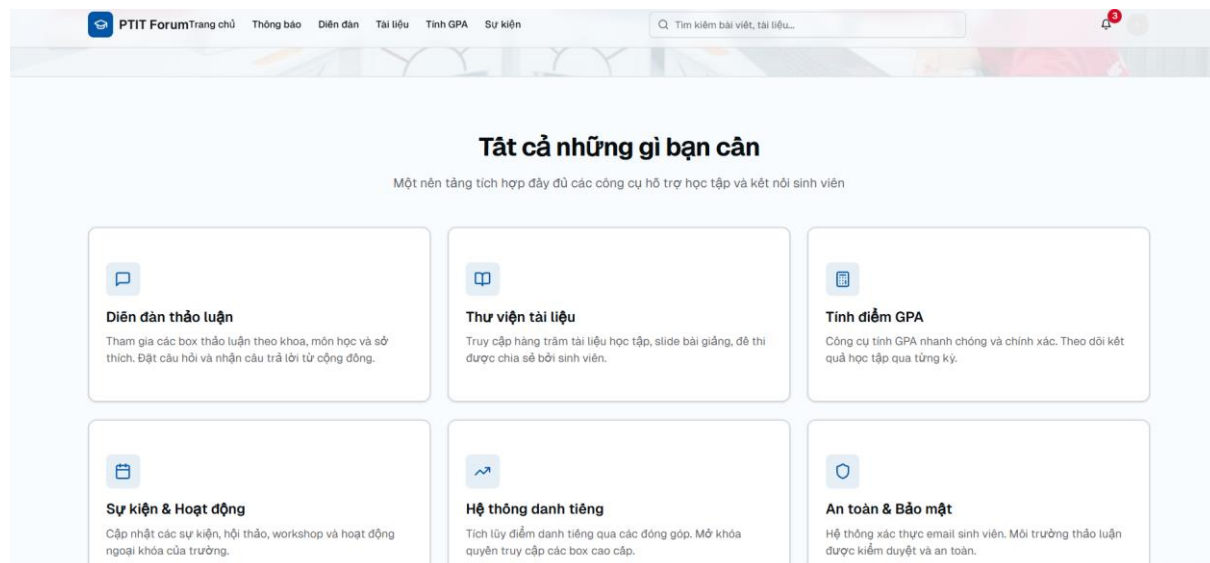
5.2 Giao diện và Chức năng chính (Kèm hình ảnh):



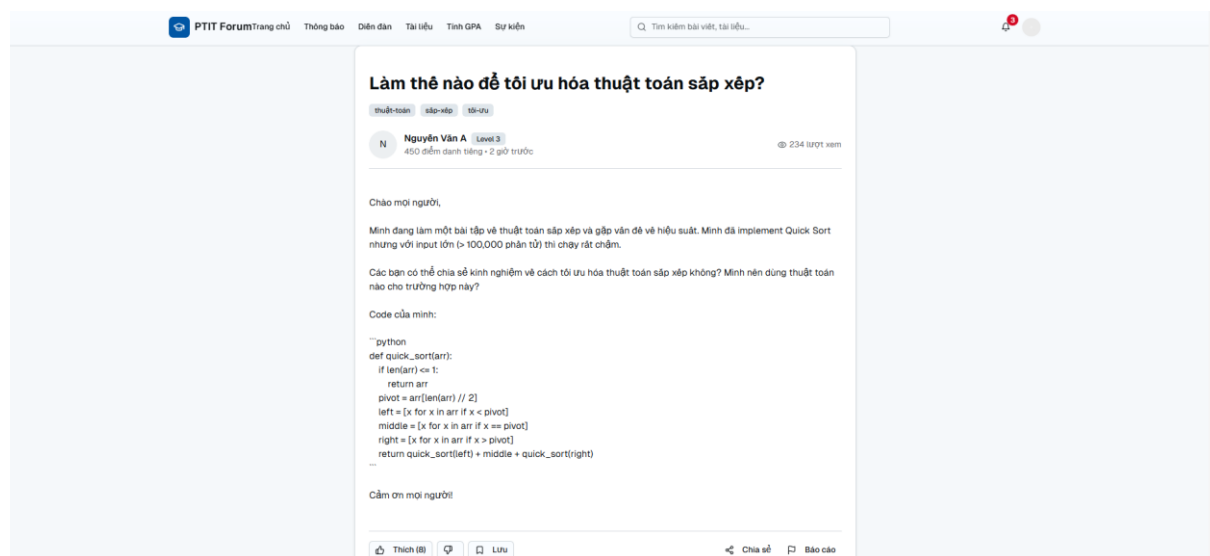
Hình 5-6: Giao diện chatbot AI



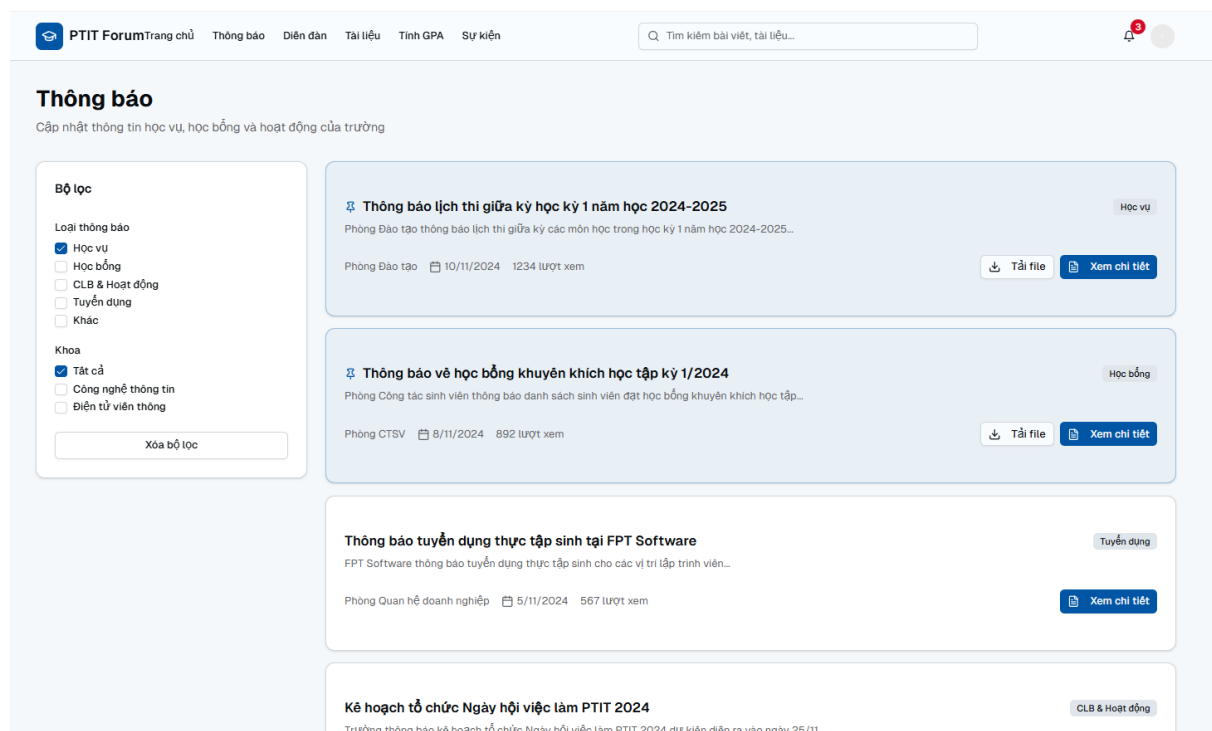
Hình 5-7: Trang chính người dùng



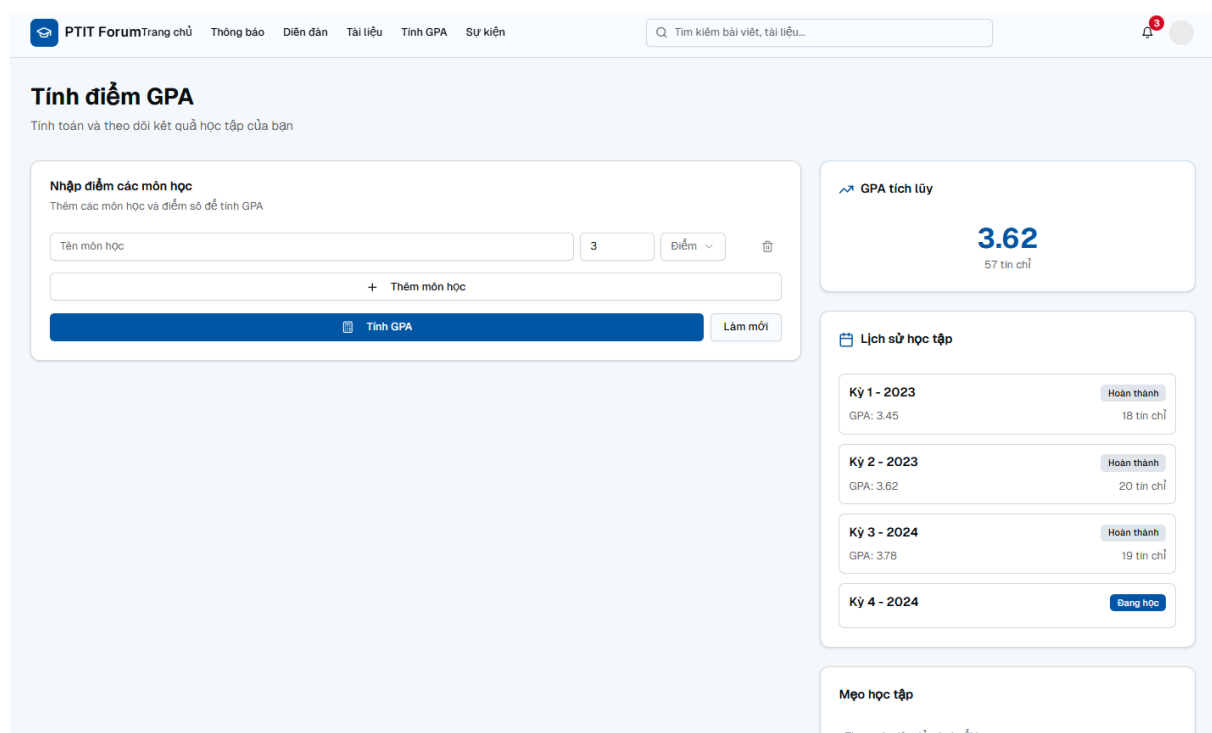
Hình 5-8: Giao diện các tính năng



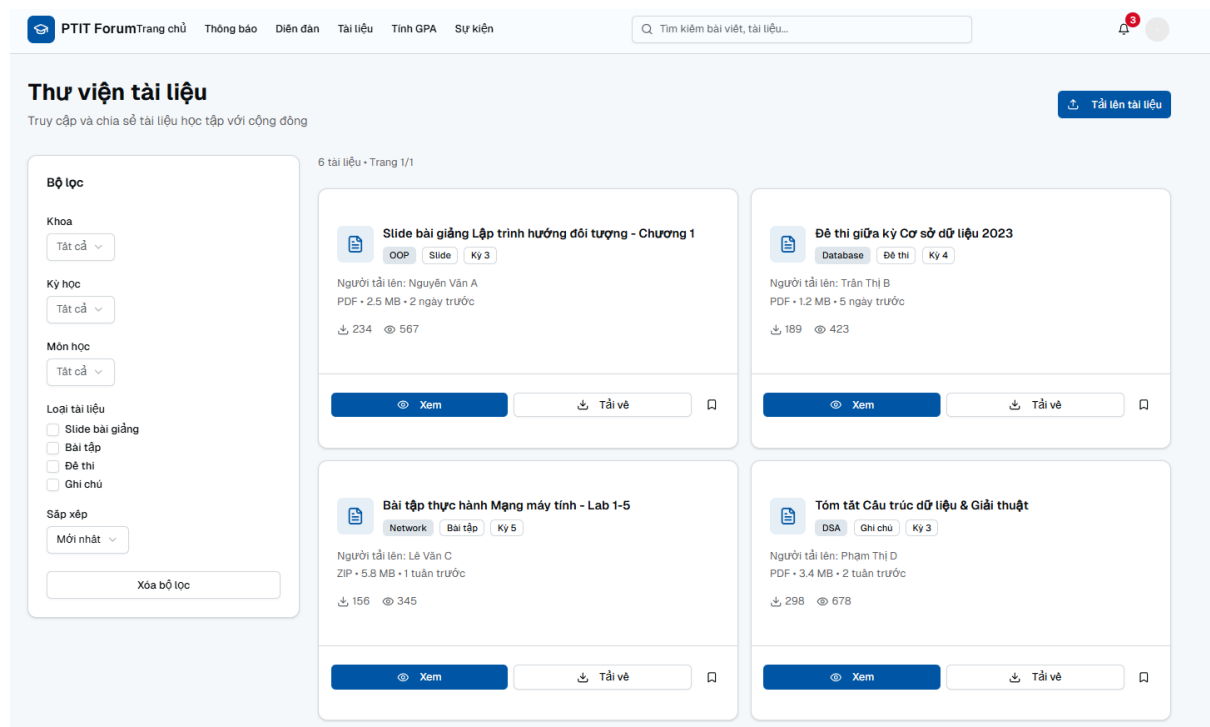
Hình 5-9: Giao diện xem bài đăng trong topic



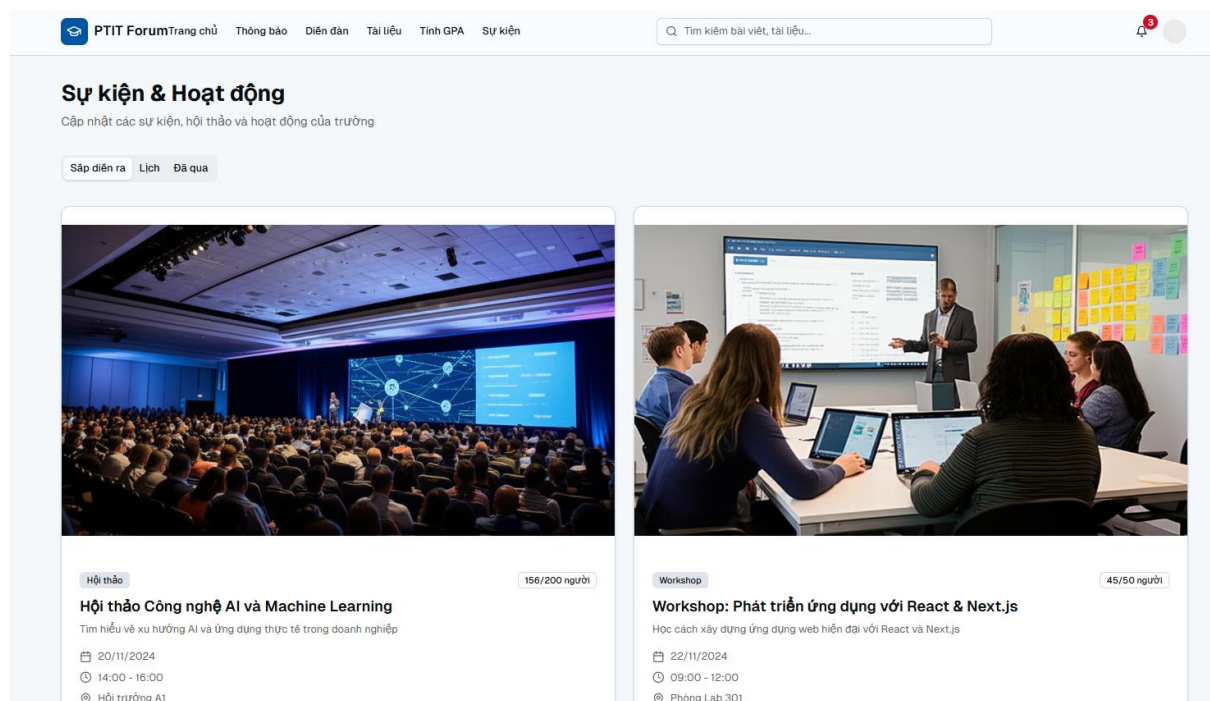
Hình 5-10: Giao diện xem thông báo



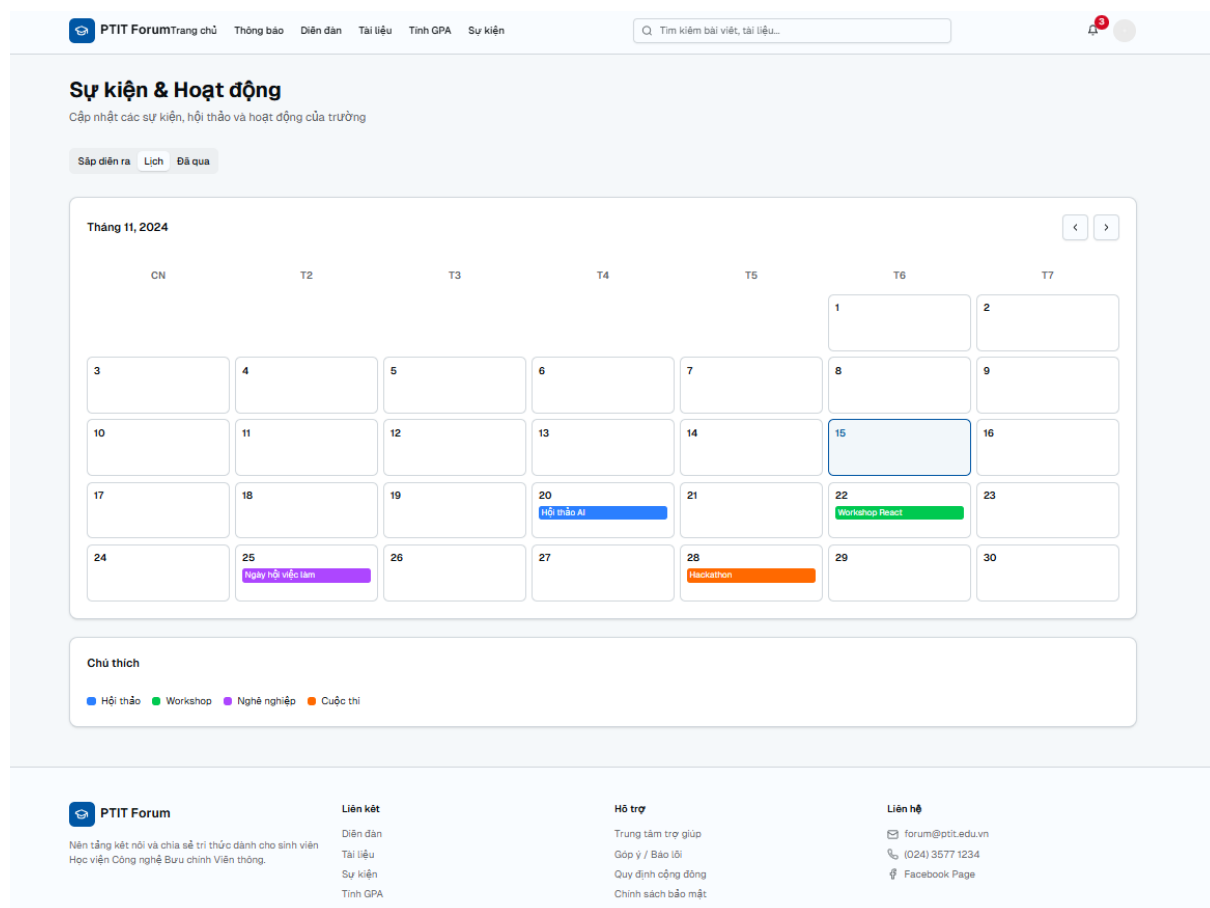
Hình 5-11: Giao diện tính điểm



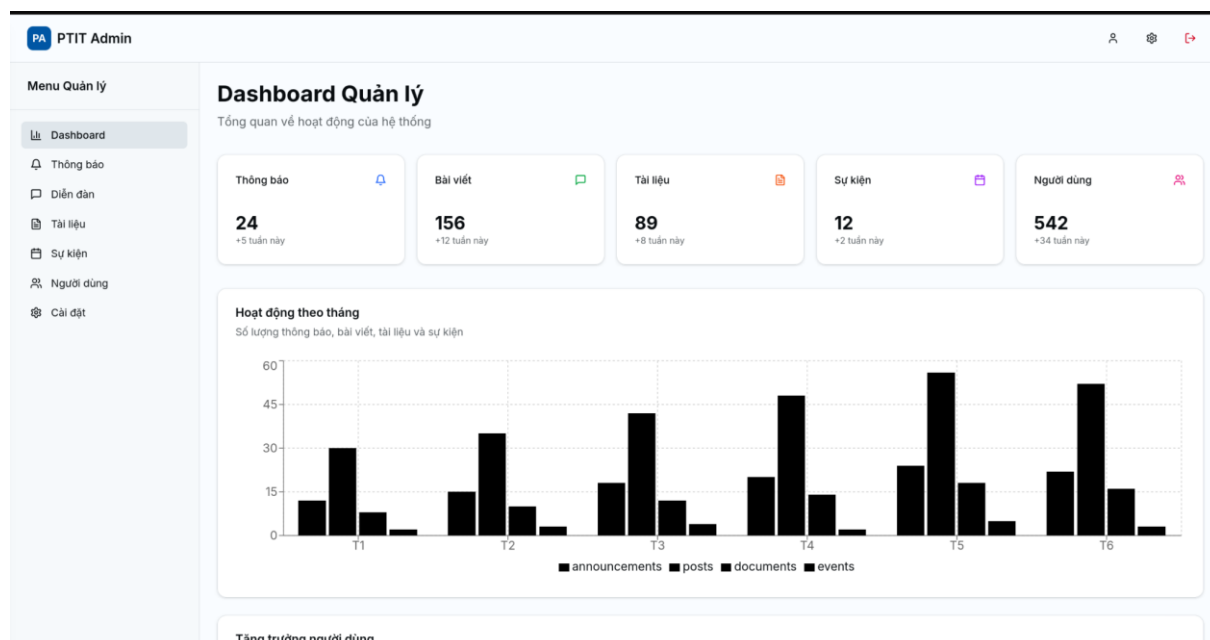
Hình 5-12: Giao diện thư viện tài liệu



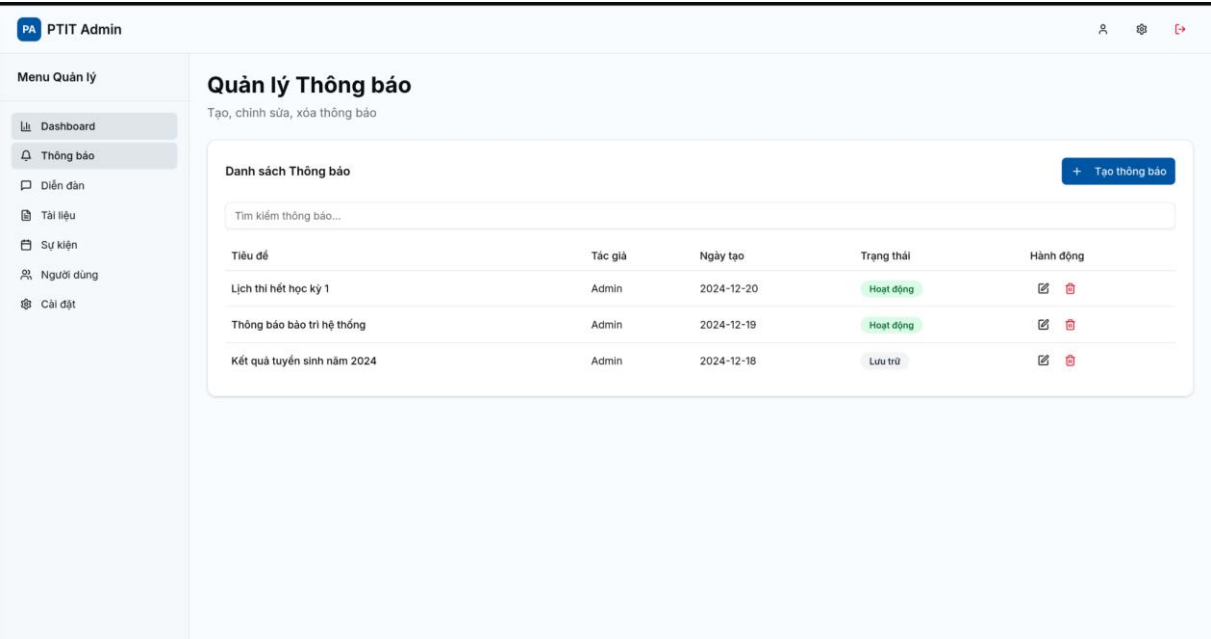
Hình 5-13: Giao diện sự kiện & hoạt động



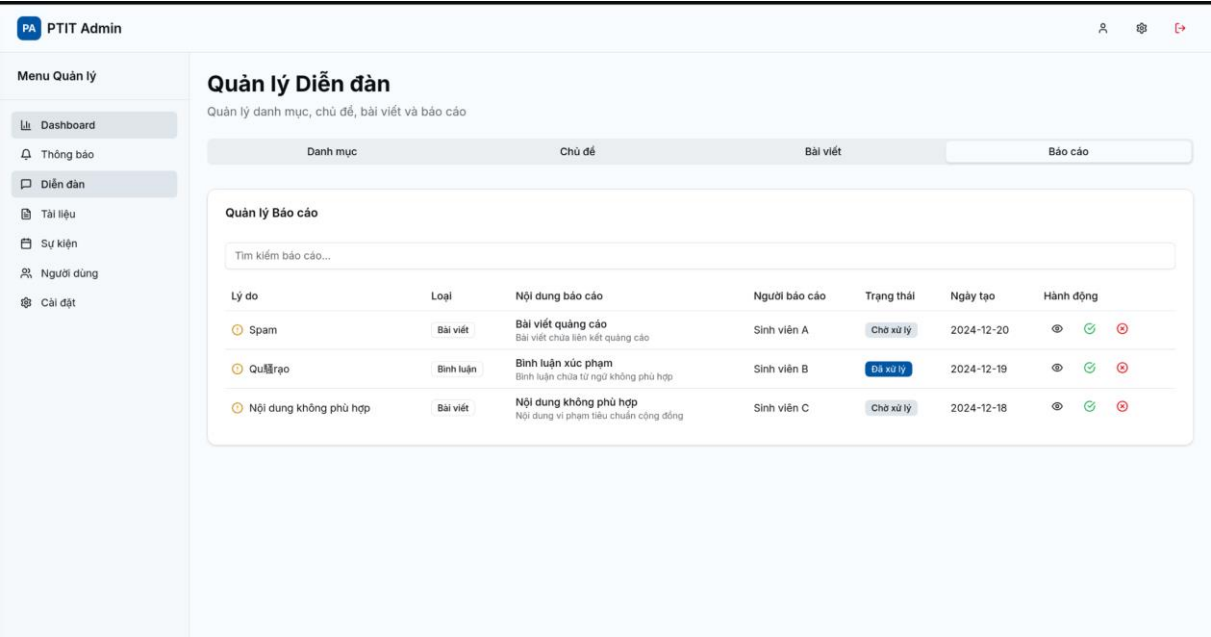
Hình 5-14: Giao diện lịch sự kiện



Hình 5-15: Giao diện admin dashboard



Hình 5-16: Giao diện quản lý thông báo



Hình 5-17: Giao diện quản lý diễn đàn

TÀI LIỆU THAM KHẢO

- [1] Spring Boot Documentation : <https://spring.io/projects/spring-boot>
- [2] Vector Similarity Explained: <https://www.pinecone.io/learn/vector-similarity/>
- [3] Flowiseai Documentation : <https://docs.flowiseai.com/>
- [4] retrieval-augmented generation (RAG) :
https://vi.wikipedia.org/wiki/T%E1%BA%A1o_sinh_d%E1%BB%B1a_tr%C3%AAn_truy_xu%E1%BA%A5t_t%C4%83ng_c%C6%B0%E1%BB%9Dng
- [5] Các mô hình LLM của Gemini Gemini: <https://ai.google.dev/gemini-api/docs?hl=vi>
- [6] Các mô hình LLM của Qwen <https://qwen.readthedocs.io/en/latest/>
- [7] Nextjs Documentation : <https://nextjs.org/docs>